ĐẠI HỌC QUỐC GIA - THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

Học sâu và ứng dụng trong thị giác máy tính (CO5085)

Bài tập

PHÂN LOẠI ẢNH VỚI CÁC MÔ HÌNH CƠ BẢN

Giảng Viên: TS Lê Thành Sách
Sinh viên: Bùi Nguyễn Gia Huy - 2570201
Nguyễn Ngọc Minh - 2570111
Lê Ngọc Minh Thư - 2570331
Phạm Minh Quang - 2570302

TP. Hồ Chí Minh, Tháng 3/2026

Mục lục

Danh sách hình vẽ **4**

Danh sách bảng **5**

1 Giới thiệu **7**

1.1 Mục tiêu bài báo cáo 7

1.2 Giới thiệu tập dữ liệu CIFAR-10 7

1.2.1 Đặc trưng dữ liệu 8

1.2.2 Phân phối dữ liệu 9

1.2.3 Trực quan hóa dữ liệu 9

1.2.4 Tiền xử lý dữ liệu 10

2 Phần 1 — Xây dựng các mô hình phân loại **12**

2.1 Softmax Regression 12

2.1.1 Giới thiệu 12

2.1.2 Xây dựng kiến trúc 12

2.1.3 Nhận xét 13

2.2 Multi-Layer Perceptron (MLP) 13

2.2.1 Giới thiệu 13

2.2.2 Xây dựng kiến trúc 13

2.2.3 Nhận xét 14

2.3 Convolutional Neural Network (ResNet50) 14

2.3.1 Giới thiệu 14

2.3.2 Xây dựng kiến trúc 15

2.3.3 Nhận xét 15

2.4 Vision Transformer (ViT) 16

2.4.1 Giới thiệu 16

2.4.2 Xây dựng kiến trúc 16

2.4.3 Nhận xét 17

3 Phần 2 — Huấn luyện, đánh giá và so sánh **18**

3.1 Thiết lập thực nghiệm 18

3.1.1 Tiền xử lý và Chiến lược tăng cường dữ liệu 18

3.1.2 Cấu hình huấn luyện đồng bộ và tùy chỉnh 18

3.1.2.1 Các thiết lập chung 18

3.1.2.2 Phân bổ Learning Rate theo đặc thù mô hình 19
3.2 Phân tích lịch sử huấn luyện 19
3.3 Đánh giá và So sánh hiệu năng trên tập Test 21
3.3.1 Kết quả định lượng trên tập dữ liệu kiểm thử 21
3.3.2 Phân tích sai số qua Ma trận nhầm lẫn 21
3.4 Kết luận 23

4 Phần 3 — Tự hiện thực TransformerEncoder và ViT 24
4.1 Mục tiêu thực nghiệm 24
4.2 Kiến trúc mô hình hiện thực từ đầu 24
4.2.1 Cấu trúc Patch Embedding và Token 24
4.2.2 Khối Transformer Encoder tùy chỉnh 25
4.3 Thiết lập huấn luyện và Benchmark 26
4.3.1 Thông số cấu hình đồng bộ 26
4.4 Kết quả thực nghiệm và Phân tích 27
4.4.1 So sánh quy mô tham số 27
4.4.2 Hiệu năng trên tập kiểm thử (Test Set) 27
4.4.3 Nhận xét và thảo luận 29
4.5 Kết luận 29

5 Phần 4 — Các kiến trúc kết hợp và cách embed ảnh khác nhau 30
5.1 Mục tiêu thực nghiệm 30
5.2 Thiết kế kiến trúc và các phương pháp Biểu diễn Token 30
5.2.1 Kiến trúc lai HybridViT (CNN + Transformer) 30
5.2.2 Các biến thể Patch Embedding (Tokenization) 31
5.2.2.1 Grid Patch Embedding (Ô lưới tiêu chuẩn) 31
5.2.2.2 Overlap Patch Embedding (Vùng chồng lấp) 31
5.2.2.3 Multi-Scale Patch Embedding (Đa tỉ lệ) 31
5.3 Quy trình huấn luyện và Đánh giá 31
5.3.1 Thiết lập thông số huấn luyện (Hyperparameters) 32
5.3.2 Kiểm soát và Lưu trữ 32
5.3.3 Các chỉ số đo lường 32
5.4 Kết quả thực nghiệm và Phân tích 32
5.4.1 Bảng thông số cấu trúc và quy mô tham số 32
5.4.2 Hiệu năng định lượng trên tập dữ liệu 33
5.4.3 Trực quan hóa kết quả thực nghiệm 33
5.5 Nhận xét và Thảo luận 33
5.5.1 Sức mạnh của Inductive Bias trong kiến trúc HybridViT 34
5.5.2 So sánh các chiến lược Patch Embedding 34

5.5.3 Đánh giá sự đánh đổi (Trade-off) tham số và thời gian 34
5.6 Kết luận 34

6 Phần 5 — Mô hình phân loại dựa trên LSTM/GRU 35
6.1 Giới thiệu 35
6.2 Phương pháp chuyển ảnh thành chuỗi 35
6.3 Kiến trúc mô hình 36
6.3.1 Cơ chế biến đổi ảnh thành chuỗi (Image-to-Sequence) 36
6.3.2 Các biến thể Classifier (LSTM và GRU) 36
6.3.3 Cấu hình mạng chuyên sâu 36
6.4 Thiết lập thực nghiệm 37
6.4.1 Siêu tham số huấn luyện (Hyperparameters) 37
6.4.2 Các mô hình so sánh 37
6.5 Kết quả thực nghiệm 38
6.5.1 Hiệu năng định lượng trên tập dữ liệu CIFAR-10 38
6.5.2 Trực quan hóa quá trình hội tụ 38
6.5.3 So sánh độ chính xác theo từng lớp đối tượng 39
6.6 Phân tích và Nhận xét kết quả 39
6.6.1 Sự vượt trội của kiến trúc GRU 39
6.6.2 Ảnh hưởng của hướng quét (Row-wise vs Column-wise) 40
6.6.3 Phân tích sự nhầm lẫn giữa các lớp động vật 40
6.7 Kết luận 40

Danh sách hình vẽ

1.1 Biểu đồ phân tích cường độ pixel theo từng kênh màu RGB 8
1.2 Sự phân phối của các lớp trong CIFAR-10 9
1.3 Hình ảnh trực quan hóa các mẫu đại diện cho mỗi lớp 9

3.1 Diễn biến các chỉ số huấn luyện và kiểm thử của 4 kiến trúc mô hình trên CIFAR-10 20
3.2 Biểu đồ so sánh hiệu năng trực quan giữa các kiến trúc thực nghiệm. . . . 21
3.3 Ma trận nhầm lẫn chi tiết cho từng mô hình phân loại. 22

4.1 Cấu trúc chi tiết một khối Transformer Encoder được hiện thực 25
4.2 So sánh lịch sử huấn luyện giữa ViT_scratch và ViT_builtin qua 20 epochs 28
4.3 Ma trận nhầm lẫn của hai mô hình trên tập Test 28

5.1 Phân tích trực quan từ thực nghiệm: (A) Lịch sử Acc/Loss; (B) So sánh Test Accuracy; (C) Tương quan Params/Acc; (D) Thời gian huấn luyện. . 33

6.1 So sánh Training Loss 38
6.2 So sánh Validation Loss 38
6.3 So sánh Validation Accuracy 39
6.4 Chỉ số F1-Score (Macro) 39
6.5 Biểu đồ so sánh độ chính xác theo từng lớp giữa các biến thể RNN 39

Danh sách bảng

3.1 Bảng thông số Learning Rate chi tiết cho từng kịch bản thực nghiệm . . . 19

4.1 So sánh số lượng tham số giữa hai phiên bản ViT 27
4.2 Kết quả định lượng của hai mô hình sau 20 epoch 27

5.1 Thống kê quy mô tài nguyên và chi phí huấn luyện 33
5.2 So sánh đầy đủ các chỉ số hiệu năng giữa các biến thể kiến trúc 33

6.1 Cấu hình siêu tham số trong quá trình thực nghiệm 37
6.2 So sánh hiệu năng chi tiết giữa các kiến trúc RNN 38

Danh sách thuật toán

1 Softmax Regression Forward Pass 13
2 MLP Forward Pass 14
3 ResNet50 Forward Pass (Optimized for CIFAR-10) 15
4 Custom Vision Transformer (SimpleViT) Forward Pass 17

1

Giới thiệu

1.1 Mục tiêu bài báo cáo

Bài tập tập trung vào việc hệ thống hóa kiến thức học sâu cơ bản và kỹ năng ứng dụng trong thị giác máy tính qua các nội dung sau:

- Xây dựng và so sánh các mô hình phân loại ảnh từ đơn giản đến phức tạp: softmax regression, MLP, CNN, Vision Transformer (ViT), và mô hình dựa trên LSTM/GRU.
- Tự viết vòng lặp huấn luyện (training loop) bằng PyTorch, sử dụng backward có sẵn để tính đạo hàm, thay vì chỉ gọi API cấp cao.
- Hiểu cấu trúc bên trong Transformer qua việc tự hiện thực khối TransformerEncoder từ các phép toán cơ bản.
- Khám phá các cách thiết kế kiến trúc khác nhau: kết hợp CNN-Transformer, các cách tokenize/embed ảnh khác nhau, và biểu diễn ảnh thành chuỗi cho RNN.

1.2 Giới thiệu tập dữ liệu CIFAR-10

Nhóm quyết định lựa chọn CIFAR-10 làm cơ sở thực nghiệm thay vì các tập dữ liệu đơn giản hơn như MNIST nhờ vào các đặc tính ưu việt sau:

- **Độ phức tạp của dữ liệu thực tế:** Khác với các chữ số viết tay đã bão hòa về độ chính xác, CIFAR-10 chứa các thực thể thực tế với sự đa dạng về màu sắc, kết cấu và bối cảnh. Các đối tượng thường không nằm chính giữa khung hình, đòi hỏi mô hình phải có khả năng trích xuất đặc trưng (*feature extraction*) mạnh mẽ.
- **Thách thức về độ phân giải:** Với kích thước giới hạn 32×32 pixel, các chi tiết nhỏ bị lược bỏ đáng kể. Điều này buộc mô hình phải học cách nhận diện dựa trên cấu trúc tổng thể và các đặc trưng bậc cao thay vì dựa vào các chi tiết cục bộ.
- **Tính cân bằng tuyệt đối giữa các lớp:** Mỗi lớp đối tượng đều có đúng 6.000 hình ảnh. Sự cân bằng này giúp loại bỏ hiện tượng thiên kiến (*bias*) trong quá trình huấn luyện, đồng thời giúp các chỉ số đánh giá như Accuracy, F1-Score và Precision trở nên khách quan hơn.

- **Tính nhất quán trong thực nghiệm:** Việc phân chia sẵn 50.000 ảnh huấn luyện và 10.000 ảnh kiểm thử theo các phân đoạn (*batches*) giúp kết quả thực nghiệm có tính ổn định, dễ dàng đối chiếu và so sánh với các nghiên cứu tiêu chuẩn khác.

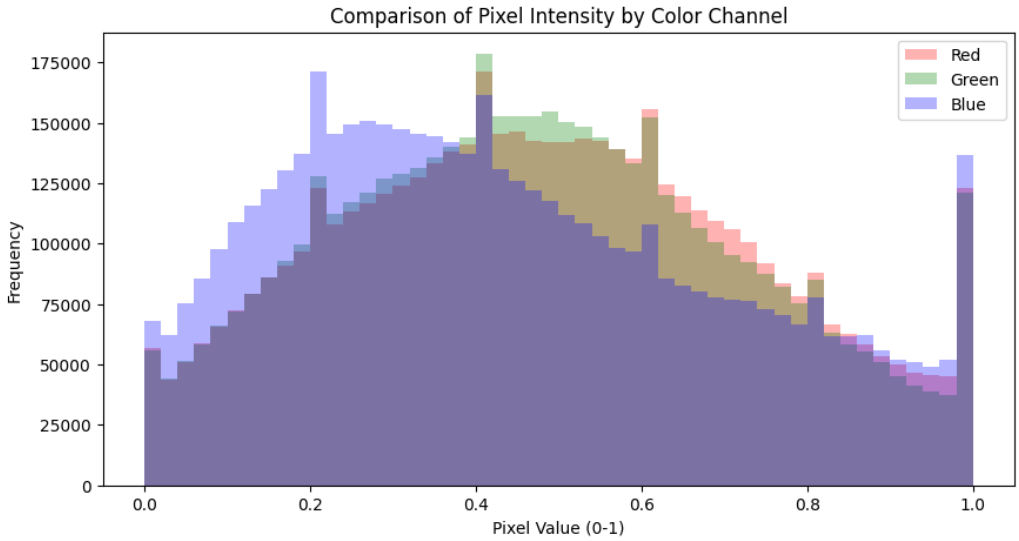
1.2.1 Đặc trưng dữ liệu

CIFAR-10 là tập dữ liệu gồm tổng cộng **60,000 ảnh màu**, mỗi ảnh có kích thước nhỏ 32×32 pixel. Tập dữ liệu được chia thành **10 lớp đối tượng**, bao gồm: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.

Về cách phân chia dữ liệu:

- **50,000 ảnh** được sử dụng cho tập huấn luyện (training set)
- **10,000 ảnh** còn lại được dùng cho tập kiểm tra (test set)

Đặc điểm nổi bật của CIFAR-10 là kích thước ảnh nhỏ nhưng chứa nhiều biến thể về góc nhìn, ánh sáng và nền, khiến bài toán phân loại trở nên không hề đơn giản. Điều này giúp tập dữ liệu trở thành một lựa chọn phù hợp để đánh giá khả năng học đặc trưng của các mô hình từ cơ bản đến nâng cao.

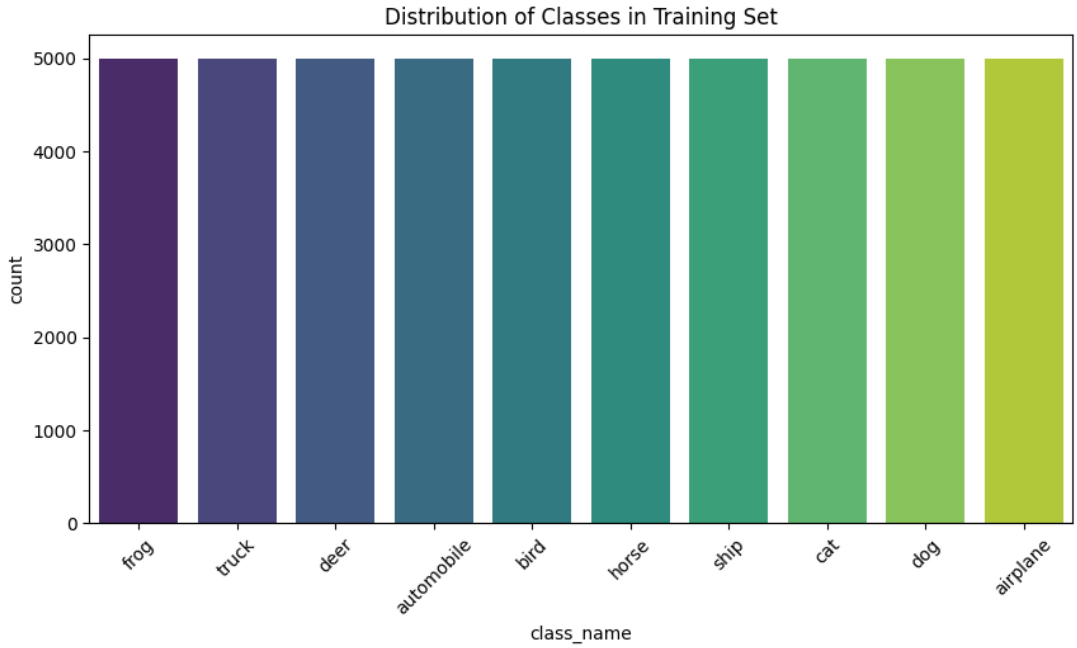


Hình 1.1: *Biểu đồ phân tích cường độ pixel theo từng kênh màu RGB*

Dựa vào biểu đồ Histogram 1.1, các giá trị pixel (sau khi chuẩn hóa về đoạn $[0, 1]$) trải dài trên toàn bộ dải giá trị. Tuy nhiên, mật độ tập trung cao nhất nằm trong khoảng $[0.2, 0.6]$, cho thấy các hình ảnh trong CIFAR-10 có độ tương phản trung bình, không quá sáng cũng không quá tối.

Mặt khác, biểu đồ còn cho thấy sự tương đồng lớn về phân phối giữa ba kênh Red, Green và Blue. Tuy nhiên, kênh Blue có xu hướng đạt đỉnh sớm hơn ở các giá trị thấp (vùng tối), trong khi kênh Red và Green có phần nhỉnh hơn ở dải giá trị cao. Điều này phản ánh sự hiện diện đáng kể của các màu sắc tự nhiên (cây cối, bầu trời, mặt nước) trong tập dữ liệu.

1.2.2 Phân phối dữ liệu



Hình 1.2: *Sự phân phối của các lớp trong CIFAR-10*

Như chúng ta đã bàn ở trước đó, CIFAR-10 thể hiện một sự phân phối đồng nhất lý tưởng. Mỗi lớp trong 10 lớp đối tượng (từ *frog* đến *airplane*) đều sở hữu chính xác 5.000 mẫu dữ liệu. Sự cân bằng này loại bỏ hoàn toàn rủi ro mô hình bị học lệch (bias) do số lượng mẫu. Điều này cho phép quá trình huấn luyện tập trung vào việc trích xuất đặc trưng phân biệt thay vì xử lý các kỹ thuật bù đắp mất cân bằng lớp (như oversampling hay undersampling).

1.2.3 Trực quan hóa dữ liệu

Hình 1.3: *Hình ảnh trực quan hóa các mẫu đại diện cho mỗi lớp*

Hình 1.3 trực quan hóa các mẫu đại diện cho thấy bản chất phức tạp của bài toán phân loại ảnh thực tế:

- Với kích thước 32×32 pixel, các đối tượng (ví dụ: *bird*, *cat*, *dog*) bị mất đi nhiều chi tiết cạnh và vân bề mặt. Điều này đòi hỏi các mô hình mạng Neural phải có khả năng học được các đặc trưng hình khối tổng thể.
- Quan sát các mẫu như *ship* hay *truck*, ta thấy sự khác biệt lớn về góc chụp, màu sắc và vùng nền phía sau. Ví dụ, lớp *airplane* có thể là máy bay trên bầu trời hoặc trên đường băng. Sự biến thiên này buộc mô hình phải có tính bất biến đối với các thay đổi về vị trí và độ sáng của đối tượng.

1.2.4 Tiền xử lý dữ liệu

Quy trình tiền xử lý dữ liệu CIFAR-10 được thể hiện qua các bước sau đây:

1. **Trích xuất và tái cấu trúc dữ liệu**

   Dữ liệu gốc từ CIFAR-10 được lưu trữ dưới dạng các file nhị phân (binary batches). Quá trình trích xuất chuyển đổi các chuỗi byte này thành các mảng số nguyên.

   Dữ liệu sau đó được tái cấu trúc về định dạng chuẩn của PyTorch là (N, C, H, W) – tương ứng với (Số lượng ảnh, Số kênh màu, Chiều cao, Chiều rộng).

   Mỗi ảnh từ mảng phẳng 3072 phần tử được định hình lại thành $(3, 32, 32)$. Việc đưa kênh màu (C) lên trước giúp tối ưu hóa việc tính toán tích chập trên các thư viện tăng tốc phần cứng như CUDA.

2. **Chuẩn hóa và mã hóa nhãn**

   Dựa trên quan sát từ biểu đồ cường độ pixel (Hình 1.1), các giá trị được chuyển đổi từ miền $[0, 255]$ sang khoảng $[0, 1]$ bằng cách chia cho 255.0. Bước này giúp giữ cho trọng số của mô hình không bị dao động quá lớn và ngăn chặn hiện tượng bùng nổ gradient (gradient explosion).

   Sau đó, nhãn của 10 lớp đối tượng được chuyển đổi từ số nguyên sang vector xác suất 10 chiều. Ví dụ, nhãn của lớp *dog* (vị trí thứ 5) sẽ là $[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]$. Điều này cho phép sử dụng hàm mất mát Cross-Entropy Loss để đo lường khoảng cách giữa dự báo của mô hình và thực tế.

3. **Tăng cường dữ liệu**

   Để cải thiện khả năng tổng quát hóa, các phương pháp điều chuẩn thông qua tăng cường dữ liệu đã được tích hợp vào quy trình huấn luyện. Nhóm áp dụng *RandomHorizontalFlip* (xác suất 50%) và *RandomCrop* (với lề đệm tương ứng để duy trì độ phân giải 32×32) nhằm giúp mạng nơ-ron nhận diện đối tượng ở các vị trí và hướng khác nhau. Ngoài ra, phép xoay ngẫu nhiên (*RandomRotation*) được sử dụng để mô phỏng các biến dạng hình học, từ đó giúp mô hình đạt được tính bất biến đối với các thay đổi nhỏ về góc xoay của dữ liệu đầu vào.

4. **Phân tách tập dữ liệu**

   Tập dữ liệu huấn luyện 50.000 ảnh được chia tách theo tỷ lệ 80/20, trong đó 40.000 ảnh là sẽ là Training Set, còn lại là Validation Set. Training Set dùng để

cập nhật trọng số cho mô hình, Validation Set dùng để tinh chỉnh siêu tham số (hyperparameters) và theo dõi Overfitting trong quá trình huấn luyện mà không can thiệp vào tập Test độc lập.

2

Phần 1 — Xây dựng các mô hình phân loại

Trong phần này, nhóm đã tiến hành xây dựng và thực nghiệm bốn mô hình phân loại trên tập dữ liệu CIFAR-10 bao gồm: Softmax Regression, MLP, ResNet50 (CNN-based) và Vision Transformer (ViT). Các mô hình được thiết kế nhằm đánh giá sự khác biệt giữa các phương pháp tiếp cận từ tuyến tính đơn giản đến các kiến trúc hiện đại dựa trên Attention.

2.1 Softmax Regression

2.1.1 Giới thiệu

Softmax Regression (còn gọi là Multinomial Logistic Regression) là mô hình phân loại tuyến tính cơ bản nhất được triển khai trong nghiên cứu này. Mô hình thực hiện ánh xạ trực tiếp từ không gian vector đặc trưng thô của ảnh sang xác suất của 10 lớp đối tượng.

Do có cấu trúc đơn giản và dễ huấn luyện, Softmax Regression thường được sử dụng làm baseline để so sánh với các mô hình phức tạp hơn.

2.1.2 Xây dựng kiến trúc

Mô hình Softmax Regression được nhóm triển khai theo biến thể "High-Stability"nhằm thiết lập mức hiệu năng baseline ổn định. Thay vì chỉ sử dụng một phép ánh xạ tuyến tính thuần túy, kiến trúc này được tích hợp thêm tầng chuẩn hóa để tối ưu hóa dữ liệu pixel thô trước khi phân loại.

- **Flatten Layer:** Sử dụng `nn.Flatten()` để biến đổi ảnh đầu vào từ tensor 4 chiều thành vector một chiều có kích thước $3 \times 32 \times 32 = 3072$.
- **Batch Normalization (1D):** Sử dụng lớp `BatchNorm1d` với 3072 tham số học được. Việc áp dụng chuẩn hóa ngay tại tầng đầu vào giúp hiệu chỉnh phân phối dữ liệu thô (raw pixel distribution), giảm thiểu hiện tượng lệch gradient (gradient shift) và đảm bảo tính ổn định trong quá trình tối ưu hóa.
- **Linear Layer:** Thực hiện phép biến đổi affine $y = XW^T + b$, trong đó ma trận trọng số $W \in \mathbb{R}^{10\times3072}$, ánh xạ từ không gian đặc trưng sang 10 logit tương ứng với 10 lớp đối tượng.

Input: Input batch X (shape: $B, 3, 32, 32$)
Output: Prediction probabilities P

```
// 1. Reshape image to 1D vector
```
$x \leftarrow \text{Flatten}(X)$; `// Output size:` $B \times 3072$

```
// 2. Apply 1D Batch Normalization for stability
```
$x \leftarrow \text{BatchNorm1d}(x)$;

```
// 3. Linear transformation (fully connected layer)
```
$z \leftarrow \text{Linear}(x, \text{out_features} = 10)$; `// z contains raw scores (logits)`

```
// 4. Convert logits to probabilities
```
$P \leftarrow \text{Softmax}(z)$;
return P;

Thuật toán 1. Softmax Regression Forward Pass

2.1.3 Nhận xét

Mô hình đạt độ chính xác 39.41% trên tập kiểm thử. Đây là mức hiệu năng giới hạn của các phép toán tuyến tính đối với dữ liệu ảnh màu phức tạp.

Nguyên nhân kỹ thuật:

- **Thiếu tính phi tuyến:** Việc không có tầng ẩn khiến mô hình không thể học được các đặc trưng hình khối phức tạp.
- **Mất cấu trúc không gian:** Thao tác làm phẳng (*Flatten*) ảnh làm mất sự liên kết giữa các pixel lân cận, khiến mô hình chỉ nhận diện dựa trên màu sắc tổng thể.
- **Hiệu quả của BatchNorm:** Kết quả này vượt mức baseline thông thường nhờ tầng **BatchNorm1d** đã giúp ổn định dữ liệu đầu vào và tối ưu hóa luồng gradient.

2.2 Multi-Layer Perceptron (MLP)

2.2.1 Giới thiệu

Multi-Layer Perceptron (MLP) mở rộng khả năng của mô hình tuyến tính bằng cách bổ sung các tầng ẩn (*Hidden Layers*) và hàm kích hoạt phi tuyến. Kiến trúc này cho phép mạng nơ-ron học được các biểu diễn đặc trưng phức tạp hơn từ dữ liệu ảnh thô của CIFAR-10.

2.2.2 Xây dựng kiến trúc

Nhóm triển khai cấu trúc với 4 hidden layers, tập trung vào việc duy trì lượng thông tin lớn ở các tầng đầu và giảm dần về phía đầu ra:

- **Cấu trúc tầng ẩn:** Các tầng có số lượng nơ-ron lần lượt là $2048 \rightarrow 1024 \rightarrow 512 \rightarrow 256$.

- **Thành phần mỗi tầng:** Kết hợp bộ ba `Linear` → `BatchNorm1d` → `ReLU`.
- **Cơ chế chống quá khớp (Regularization):** Tích hợp các lớp **Dropout** với tỉ lệ từ 0.3 đến 0.4 tại các tầng rộng để tăng khả năng tổng quát hóa của mô hình.

Input: $X \in \mathbb{R}^{B\times 3\times 32\times 32}$
Output: $P \in \mathbb{R}^{B\times 10}$

$x \leftarrow \text{Flatten}(X)$;

`// Iterative deep hidden layers`
foreach dim, p *in* $\{(2048, 0.4), (1024, 0.4), (512, 0.3), (256, 0.0)\}$ **do**
  $x \leftarrow \text{Linear}(x, dim) \rightarrow \text{BatchNorm1d}(x) \rightarrow \text{ReLU}(x)$;
  if $p > 0$ **then**
    $x \leftarrow \text{Dropout}(x, p)$;
  end
end

$z \leftarrow \text{Linear}(x, 10)$;
$P \leftarrow \text{Softmax}(z)$;
return P;

Thuật toán 2. MLP Forward Pass

2.2.3 Nhận xét

Mô hình MLP đạt hiệu năng cải thiện rõ rệt so với Softmax Regression (Accuracy: 58.88%) nhờ các yếu tố:

- **Độ sâu và độ rộng:** Việc sử dụng tầng ẩn lên đến 2048 nơ-ron cung cấp không gian tham số đủ lớn để xấp xỉ các đặc trưng phi tuyến phức tạp.
- **Hiệu quả của Dropout và Label Smoothing:** Các kỹ thuật này giúp kiểm soát hiện tượng quá khớp (*overfitting*) thường gặp ở các mạng kết nối đầy đủ có số lượng tham số lớn.
- **Hạn chế cốt lõi:** Dù kiến trúc sâu, MLP vẫn xử lý dữ liệu dưới dạng vector phẳng, dẫn đến việc bỏ qua các tương quan không gian giữa các pixel - một đặc tính quan trọng của dữ liệu ảnh.

2.3 Convolutional Neural Network (ResNet50)

2.3.1 Giới thiệu

Mạng nơ-ron tích chập (CNN) được thiết kế để khai thác cấu trúc không gian 2D và các tính chất cục bộ của hình ảnh. Trong nghiên cứu này, kiến trúc ResNet50 được lựa chọn làm đại diện nhờ vào cơ chế học dư (*Residual Learning*). Thông qua các kết nối tắt (*shortcut connections*), ResNet50 giải quyết hiệu quả bài toán suy giảm đạo hàm (*vanishing gradient*) thông qua cơ chế skip connection.

2.3.2 Xây dựng kiến trúc

Mô hình được tùy chỉnh lại (*CIFAR-optimized*) để phù hợp với đặc thù ảnh độ phân giải thấp (32×32), thay vì sử dụng cấu trúc mặc định cho ảnh ImageNet (224×224):

- **Sửa đổi tầng đầu vào:** Thay thế tầng tích chập 7×7 bằng `Conv2d` có *kernel_size=3*, *stride=1* và *padding=1* để bảo toàn chi tiết vật thể nhỏ.
- **Loại bỏ Max-pooling:** Sử dụng `nn.Identity()` thay cho tầng *Max-pool* mặc định nhằm ngăn chặn việc giảm kích thước bản đồ đặc trưng quá sớm.
- **Khối dư (Residual Stages):** Sử dụng cấu trúc *Bottleneck* ba tầng tích chập $(1 \times 1, 3 \times 3, 1 \times 1)$ kết hợp cộng dồn *identity*, giúp truyền dẫn thông tin xuyên suốt 50 tầng mạng.
- **Classification Head:** Tầng *Global Average Pooling* kết hợp với một lớp tuyến tính để ánh xạ đặc trưng sang 10 lớp đối tượng.

Input: $X \in \mathbb{R}^{B \times 3 \times 32 \times 32}$
Output: $P \in \mathbb{R}^{B \times 10}$

`// 1. Cifar-specific Initial Conv (No Maxpool)`
$x \leftarrow \text{Conv2d}(X, \text{k} = 3, \text{s} = 1) \rightarrow \text{BN} \rightarrow \text{ReLU}$;
$x \leftarrow \text{Identity}(x)$;

`// 2. Deep Residual Learning`
foreach *Residual Block* **do**
  $shortcut \leftarrow x$;
  $x \leftarrow \text{BottleneckLayers}(x)$;
  $x \leftarrow \text{ReLU}(x + shortcut)$;
end

`// 3. Global Aggregation & Classification`
$x \leftarrow \text{GlobalAvgPool}(x) \rightarrow \text{Linear}(x, 10)$;
$P \leftarrow \text{Softmax}(x)$;
return P;

Thuật toán 3. ResNet50 Forward Pass (Optimized for CIFAR-10)

2.3.3 Nhận xét

Thực nghiệm cho thấy ResNet50 mang lại sự nhảy vọt về hiệu năng nhờ các đặc tính kỹ thuật:

- **Bảo toàn cấu trúc không gian:** Việc sử dụng các bộ lọc tích chập cho phép mô hình học được các đặc trưng hình khối (cạnh, nét) thay vì chỉ dựa vào màu sắc như cách làm phẳng ảnh của MLP.
- **Tính thích nghi cao:** Việc loại bỏ *Max-pooling* và giảm kích thước bộ lọc đầu vào giúp mô hình không bị mất dấu vật thể do độ phân giải thấp, từ đó hội tụ nhanh hơn mặc dù số lượng epoch huấn luyện ngắn.

2.4 Vision Transformer (ViT)

2.4.1 Giới thiệu

Khác với các kiến trúc tích chập truyền thống dựa trên các bộ lọc cục bộ, Vision Transformer (ViT) ứng dụng cơ chế tự chú ý (*Self-Attention*) để xử lý dữ liệu ảnh. Thay vì quét qua từng vùng nhỏ bằng kernel, ViT coi hình ảnh như một chuỗi các phân mảnh (*patches*), cho phép mô hình thiết lập các mối quan hệ ngữ nghĩa toàn cục giữa các vùng khác nhau trong ảnh ngay từ những lớp đầu tiên.

2.4.2 Xây dựng kiến trúc

Nhóm triển khai mô hình ViT tùy chỉnh (SimpleViT) được tối ưu hóa cho ảnh kích thước nhỏ (32×32) của bộ dữ liệu CIFAR-10, thay vì sử dụng các mô hình tiền huấn luyện quá lớn. Cấu trúc chi tiết bao gồm:

- **Patch Embedding (Conv2d):** Nhóm sử dụng lớp tích chập với kích thước nhân (*kernel*) và bước nhảy (*stride*) cùng bằng 4×4. Thao tác này giúp chia ảnh thành 64 phân mảnh không chồng lấn (*non-overlapping patches*), sau đó ánh xạ mỗi phân mảnh vào không gian đặc trưng 128 chiều (`embed_dim=128`).
- **Learnable Positional Encoding:** Do cơ chế Attention không có tính thứ tự, nhóm bổ sung một bộ tham số có thể học (*learnable parameters*) vào chuỗi các patches. Đặc biệt, nhóm thêm một **CLS Token** vào đầu chuỗi để đóng vai trò là đại diện tóm tắt thông tin toàn ảnh cho nhiệm vụ phân loại.
- **Bộ mã hóa Transformer (Transformer Encoder):** Sử dụng 6 tầng mã hóa (`num_layers=6`). Mỗi tầng bao gồm:
  - **Multi-Head Attention (4 heads):** Giúp mô hình tập trung vào nhiều đặc trưng khác nhau đồng thời.
  - **Pre-Layer Normalization:** Chuẩn hóa dữ liệu trước khi đưa vào các khối tính toán, giúp quá trình huấn luyện từ đầu (*from scratch*) ổn định hơn.
  - **MLP Block:** Mở rộng không gian đặc trưng lên 256 chiều trước khi nén lại để trích xuất các đặc trưng phi tuyến tính bậc cao.
- **Tầng phân loại (Classification Head):** Sau khi đi qua bộ mã hóa, chỉ có trạng thái của *CLS Token* được trích xuất và đưa qua một lớp chuẩn hóa (*LayerNorm*) cùng một lớp tuyến tính cuối cùng để đưa ra dự báo cho 10 lớp đối tượng.

Input: $X \in \mathbb{R}^{B\times3\times32\times32}$
Output: $P \in \mathbb{R}^{B\times10}$

```
// 1. Patch Partitioning and Embedding
```
$x_p \leftarrow \text{Conv2d}(X, \text{kernel=4, stride=4})$; `// Shape: (B, 128, 8, 8)`
$x_{seq} \leftarrow \text{Flatten}(x_p)$; `// Shape: (B, 64, 128)`

```
// 2. Tokenization and Positional Encoding
```
$x_{token} \leftarrow [x_{CLS}; x_{seq}]$; `// Append CLS token to sequence`
$x_{input} \leftarrow x_{token} + E_{pos}$; `//` E_{pos} `is learnable Position Embedding`

```
// 3. Transformer Encoder (6 layers)
```
for $i = 1$ **to** 6 **do**
  $x_{input} \leftarrow \text{Self-Attention}(\text{LayerNorm}(x_{input})) + x_{input}$;
  $x_{input} \leftarrow \text{MLP}(\text{LayerNorm}(x_{input})) + x_{input}$;
end

```
// 4. Classification Head
```
$z \leftarrow \text{Linear}(\text{LayerNorm}(x_{input}[0]), 10)$; `// Use CLS token output`
$P \leftarrow \text{Softmax}(z)$;
return P;

Thuật toán 4. Custom Vision Transformer (SimpleViT) Forward Pass

2.4.3 Nhận xét

Việc triển khai kiến trúc Vision Transformer trên tập dữ liệu CIFAR-10 mang lại những quan sát kỹ thuật quan trọng về cách tiếp cận phi tích chập:

- Khác với CNN vốn bị giới hạn bởi trường thụ cảm (receptive field) của các hạt nhân tích chập, cơ chế Self-Attention cho phép các phân mảnh (patches) tương tác trực tiếp với nhau ngay từ lớp đầu tiên. Điều này giúp mô hình nắm bắt được cấu trúc tổng thể của vật thể dù độ phân giải ảnh chỉ dừng lại ở mức 32×32
- Dù không sử dụng các trọng số pretrained để đảm bảo tính khách quan trong việc đánh giá kiến trúc tự xây dựng, mô hình vẫn đạt được sự hội tụ ổn định. Tuy nhiên, kết quả thực nghiệm cũng chỉ ra rằng để ViT thực sự vượt qua ResNet50 trên các tập dữ liệu quy mô nhỏ như CIFAR-10, cần một chiến lược regularization khắt khe hơn để bù đắp cho sự thiếu hụt của các định kiến cảm nhận (inductive bias) về tính bất biến không gian vốn có của CNN.

3

Phần 2 — Huấn luyện, đánh giá và so sánh

3.1 Thiết lập thực nghiệm

3.1.1 Tiền xử lý và Chiến lược tăng cường dữ liệu

Quy trình xử lý dữ liệu được thiết lập đồng bộ cho tất cả các kiến trúc nhằm tối ưu hóa khả năng học đặc trưng và giảm thiểu hiện tượng quá khớp (*overfitting*):

- **Chuẩn hóa sơ bộ (Scaling):** Dữ liệu ảnh thô được chuyển đổi từ dải pixel $[0, 255]$ sang $[0, 1]$ bằng cách ép kiểu sang `float` và chia cho 255.0.
- **Phân tách dữ liệu (Data Splitting):** Tập huấn luyện gốc được chia theo tỷ lệ 80 : 20 bằng hàm `random_split` (seed 42), thu được 40.000 mẫu huấn luyện (*Train*) và 10.000 mẫu kiểm định (*Validation*).
- **Tăng cường dữ liệu (Data Augmentation):** Để mô hình bền vững hơn với các biến thể hình học, Nhóm áp dụng chuỗi biến đổi trên tập huấn luyện bao gồm: lật ảnh ngẫu nhiên (*Horizontal Flip*), cắt ngẫu nhiên (*Random Crop* với padding=4) và xoay ảnh ngẫu nhiên (15°).
- **Chuẩn hóa sâu (Standardization):** Sử dụng `Normalize` với $mean = 0.5$ và $std = 0.5$ cho cả 3 kênh màu, đưa dữ liệu về dải quanh giá trị 0 (khoảng $[-1, 1]$) để tăng tốc độ hội tụ.

3.1.2 Cấu hình huấn luyện đồng bộ và tùy chỉnh

Để đảm bảo tính khách quan và đạt được hiệu năng tối ưu cho từng loại kiến trúc, nhóm thiết lập khung thực nghiệm kết hợp giữa các tham số cố định và các điều chỉnh riêng biệt cho từng mô hình:

3.1.2.1 Các thiết lập chung

Tất cả các mô hình trong báo cáo (Softmax, MLP, ResNet50, SimpleViT) đều được huấn luyện dựa trên bộ quy tắc chung sau:

- **Số lượng Epochs:** 20 epochs .

- **Dữ liệu đầu vào:** Sử dụng `batch_size=64` thông qua lớp bọc `AugmentedDataset` để áp dụng các kỹ thuật tăng cường dữ liệu.
- **Kỹ thuật điều tiết:** *Weight Decay* $1e-4$ và *Label Smoothing* hệ số 0.1.
- **Điều kiện lưu trữ:** Hệ thống tự động theo dõi và lưu lại bộ trọng số giúp mô hình đạt chỉ số **F1-score** cao nhất trên tập Validation.
- **Môi trường:** GPU (T4).

3.1.2.2 Phân bổ Learning Rate theo đặc thù mô hình

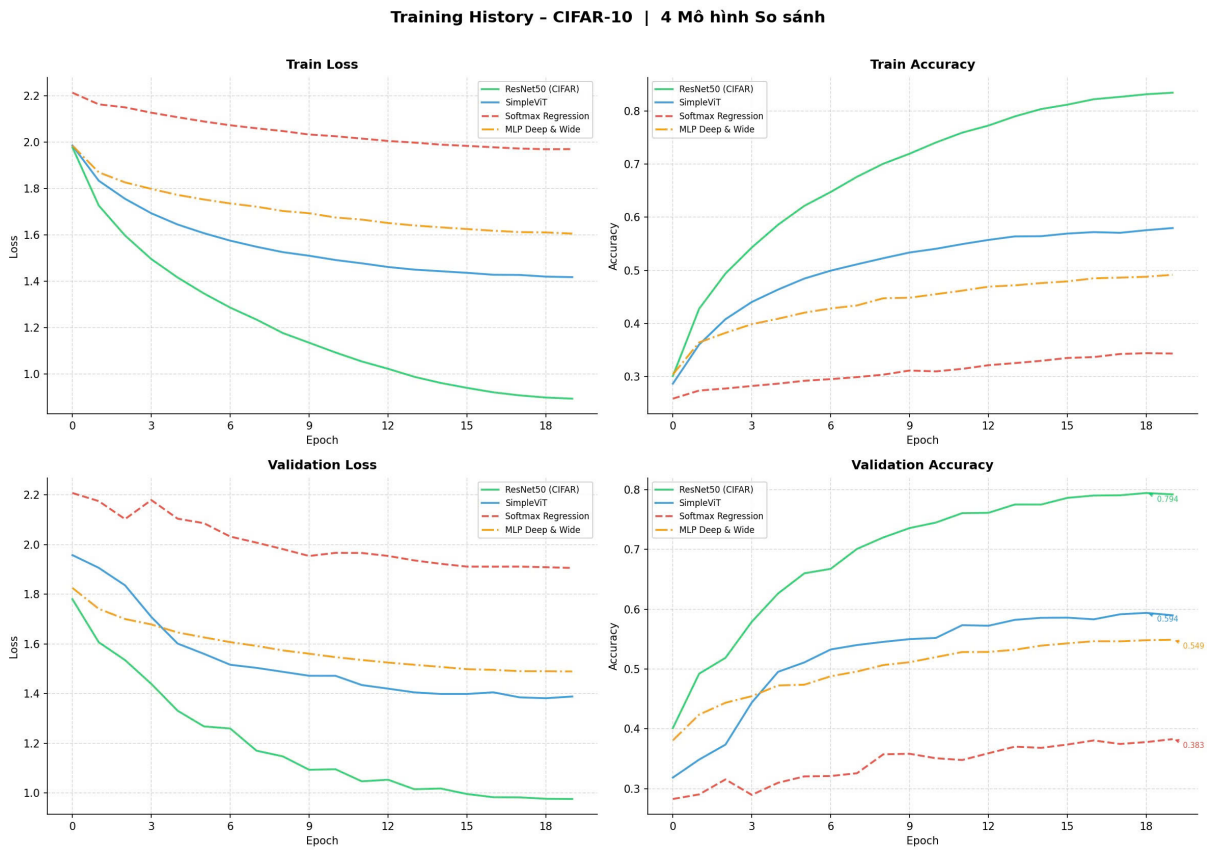
Do sự khác biệt lớn về bản chất kiến trúc và độ sâu của các mô hình , nhóm đã áp dụng các mức Learning Rate (LR) khác nhau dựa trên các quan sát sơ bộ (heuristic) để đảm bảo tất cả các mô hình có thể hội tụ trong giới hạn hẹp là 20 epochs. Nhóm thừa nhận rằng việc không sử dụng một mức LR cố định hoặc tìm kiếm ra bộ tham số tốt nhất cho từng mô hình có thể tạo ra những sai số nhất định khi so sánh hiệu năng giữa các kiến trúc. Tuy nhiên, các thiết lập hiện tại được nhóm lựa chọn nhằm ưu tiên tính ổn định của gradient trong các mạng có không gian tham số lớn

Kiến trúc mô hình	**Learning Rate**	**Lý do điều chỉnh**
Softmax Regression	1×10^{-3}	Cấu trúc đơn giản, hội tụ nhanh.
MLP	5×10^{-4}	Ổn định các tầng ẩn phi tuyến.
CNN-based	1×10^{-4}	Kiểm soát gradient trong mạng sâu.
Simple ViT	1×10^{-4}	Tối ưu cơ chế Self-Attention.

Bảng 3.1: *Bảng thông số Learning Rate chi tiết cho từng kịch bản thực nghiệm*

3.2 Phân tích lịch sử huấn luyện

Để đánh giá năng lực học tập và độ ổn định của các kiến trúc khác nhau, nhóm tiến hành theo dõi các chỉ số quan trọng qua 20 epochs. Biểu đồ tổng hợp dưới đây (Hình 3.1) so sánh diễn biến của 4 mô hình: Softmax Regression, MLP, ResNet50 và SimpleViT.



Hình 3.1: *Diễn biến các chỉ số huấn luyện và kiểm thử của 4 kiến trúc mô hình trên CIFAR-10*

Thông qua diễn biến của các đường cong huấn luyện và kiểm thử trong 20 epochs (Hình 3.1), nhóm đưa ra các đánh giá tổng quát sau:

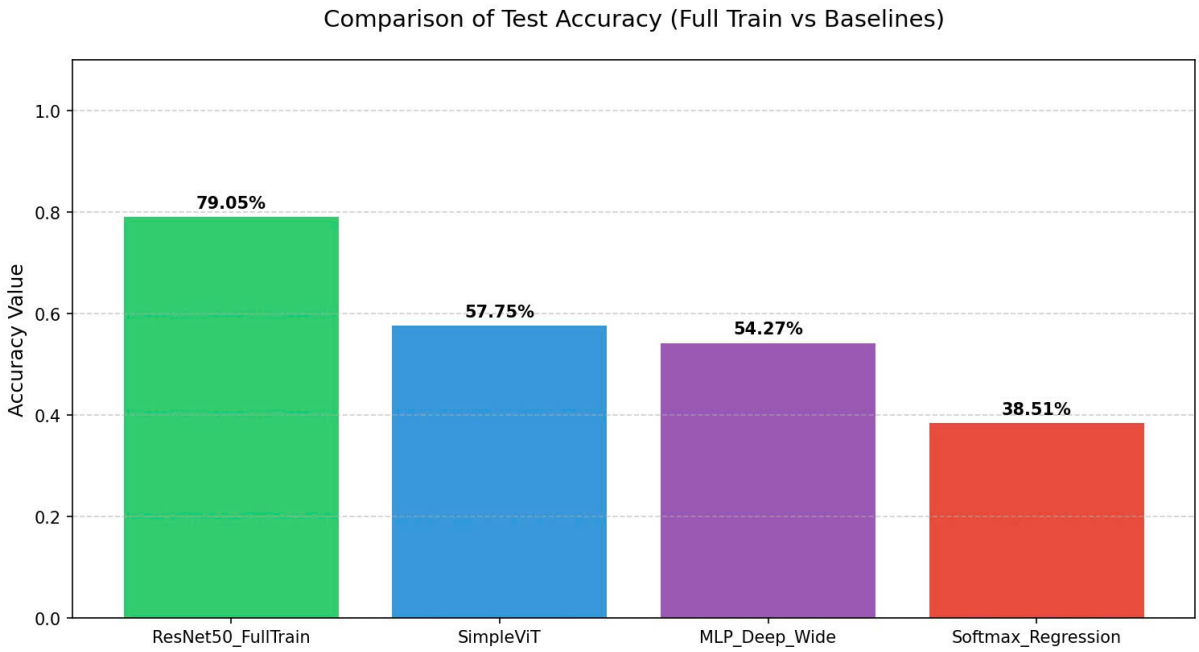
- **ResNet50** thể hiện ưu thế vượt trội với tốc độ giảm hàm mất mát nhanh nhất và đạt độ chính xác cao nhất (tiệm cận 80%). **SimpleViT** và **MLP** có tiến trình hội tụ ổn định ở mức trung bình, trong khi **Softmax Regression** bão hòa sớm và duy trì hiệu năng thấp nhất xuyên suốt quá trình.
- Các đường cong độ chính xác và hàm mất mát trên tập kiểm thử của cả 4 kiến trúc đều tiến triển hội tụ mượt mà, không xảy ra hiện tượng dao động mạnh hay phân kỳ. Điều này xác nhận việc thiết lập tốc độ học (*Learning Rate*) riêng biệt cho từng nhóm mô hình là phù hợp với đặc thù cấu tạo và quy mô gradient của chúng.

Tuy nhiên, thông qua diễn biến của các đường cong huấn luyện, nhóm nhận thấy các mô hình (đặc biệt là **MLP** và **SimpleViT**) vẫn đang trong quá trình hội tụ ổn định và chưa có dấu hiệu bão hòa hoàn toàn trong giới hạn 20 *epochs*. Việc hàm mất mát trên tập kiểm định liên tục giảm đồng thời với tập huấn luyện cho thấy các kỹ thuật điều tiết như *Weight Decay* và *Label Smoothing* đã giúp ổn định hóa quá trình cập nhật trọng số. Ngoài ra, do giới hạn về thời gian huấn luyện, các mô hình vẫn đang ở giai đoạn *underfitting* và chưa chạm tới ngưỡng *overfitting*, do đó ta có thể cải thiện độ chính xác nếu tăng số lượng *epochs* lên hàng trăm để các mô hình thực sự "học"hết các đặc trưng.

3.3 Đánh giá và So sánh hiệu năng trên tập Test

3.3.1 Kết quả định lượng trên tập dữ liệu kiểm thử

Nhóm tiến hành đánh giá năng lực phân loại của 4 kiến trúc dựa trên chỉ số Độ chính xác (*Accuracy*) đo đạc trực tiếp trên tập Test của CIFAR-10. Kết quả được tổng hợp trong Hình 3.2:

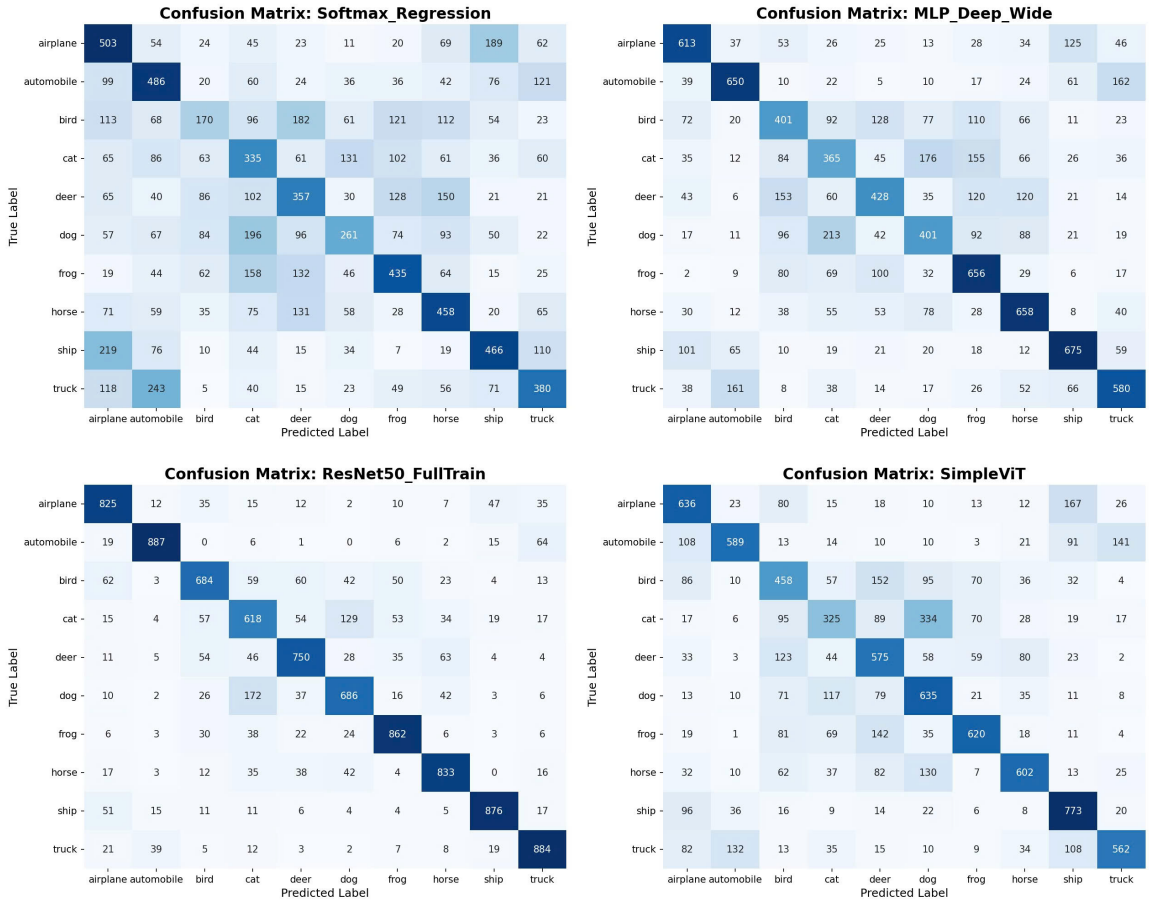


Hình 3.2: *Biểu đồ so sánh hiệu năng trực quan giữa các kiến trúc thực nghiệm.*

Biểu đồ cho thấy sự chênh lệch rõ rệt về hiệu năng giữa các mô hình, trong đó ResNet50 vượt trội hoàn toàn với độ chính xác cao nhất đạt 79.05%. Các kiến trúc học sâu hiện đại như SimpleViT (57.75%) và MLP (54.27%) dù có độ phức tạp nhất định nhưng vẫn thấp hơn đáng kể so với ResNet50, cho thấy cấu trúc Residual Learning của ResNet đặc biệt hiệu quả trong việc khai thác đặc trưng dữ liệu của bài toán này. Trong khi đó, mô hình Softmax chỉ đạt 38.51%, minh chứng rằng các mô hình tuyến tính đơn giản không đủ khả năng phân loại các tập dữ liệu có tính phi tuyến tính cao. Kết quả này khẳng định việc đầu tư vào các kiến trúc CNN sâu và được huấn luyện đầy đủ là hướng đi tối ưu nhất để tối ưu hóa độ chính xác cho hệ thống.

3.3.2 Phân tích sai số qua Ma trận nhầm lẫn

Thông qua việc quan sát ma trận nhầm lẫn của 4 mô hình (Hình 3.3), nhóm nhận thấy các xu hướng phân loại đặc thù sau:



Hình 3.3: *Ma trận nhầm lẫn chi tiết cho từng mô hình phân loại.*

- **Sự vượt trội của cấu trúc tích chập (CNN):** ResNet50 cho thấy đường chéo chính rất đậm nét, đặc biệt là ở các lớp phương tiện vận tải như *automobile* (887), *ship* (876) và *truck* (884). Điều này chứng minh các bộ lọc tích chập cực kỳ nhạy bén với các đường nét cấu trúc cứng và đặc trưng hình học của máy móc.

- **Thách thức từ nhóm lớp động vật:** Cặp lớp *Cat* - *Dog* vẫn là nguồn gốc gây sai số lớn nhất cho cả 4 kiến trúc. Ngay cả với ResNet50, vẫn có sự nhầm lẫn đáng kể giữa hai lớp này (ví dụ: 172 mẫu *cat* bị đoán nhầm thành *dog*). Trong không gian ảnh độ phân giải thấp 32×32, sự tương đồng về màu sắc và kết cấu bề mặt của động vật là một thách thức lớn đối với khả năng trích xuất đặc trưng của mọi mô hình.

- **Giới hạn của các mô hình cơ bản:**
  - **Softmax Regression:** Ma trận nhầm lẫn phân tán rộng, đặc biệt nhầm lẫn nặng nề ở lớp *airplane* với *ship* (189 mẫu), cho thấy mô hình tuyến tính không thể phân biệt được các đối tượng có cùng phông nền màu xanh (trời/biển).
  - **MLP:** Cải thiện khả năng hội tụ trên đường chéo chính so với Softmax nhưng vẫn gặp khó khăn trong việc nhận diện vật thể có tính biến đổi hình dạng cao như *bird* hay *cat*.

3.4 Kết luận

Thực nghiệm khẳng định một sự phân cấp rõ rệt về hiệu năng dựa trên bản chất kiến trúc thay vì sự khác biệt trong cấu hình huấn luyện. Việc thiết lập tiền xử lý đồng bộ và tăng cường dữ liệu đa dạng đã giúp ResNet50 giữ vững vị thế là giải pháp tối ưu nhất cho bài toán phân loại ảnh kích thước nhỏ trong điều kiện thực nghiệm hiện tại.

4

Phần 3 — Tự hiện thực TransformerEncoder và ViT

4.1 Mục tiêu thực nghiệm

Phần này tập trung vào việc kiểm chứng khả năng xây dựng cấu trúc Transformer từ các phép toán cơ bản thay vì sử dụng các module tích hợp sẵn của thư viện. Mục tiêu cụ thể bao gồm:

- Tự hiện thực khối **TransformerEncoder** bằng các lớp cơ bản (`Linear`, `Dropout`) và các phép toán ma trận.
- Sử dụng **Einsum** để tính toán cơ chế Multi-Head Attention nhằm tối ưu hóa hiệu suất và độ chính xác về mặt toán học.
- Đánh giá so sánh định lượng giữa mô hình tự xây dựng (*ViT_scratch*) và mô hình sử dụng hàm thư viện (*ViT_builtin*).

4.2 Kiến trúc mô hình hiện thực từ đầu

Kiến trúc mô hình được xây dựng dựa trên nguyên lý của Vision Transformer (ViT) nguyên bản nhưng được tùy chỉnh để xử lý dữ liệu ảnh kích thước thấp (32×32) của CIFAR-10.

4.2.1 Cấu trúc Patch Embedding và Token

Thay vì sử dụng các đặc trưng tích chập sâu, mô hình chia ảnh thành các vùng không chồng lấp:

- **Patch Embedding:** Ảnh đầu vào $\mathbf{x} \in \mathbb{R}^{3\times32\times32}$ được chia thành N patches kích thước $P \times P$ ($P = 4$). Qua lớp `nn.Conv2d` với `stride=4`, các patch này được nhúng vào không gian vector có số chiều $d_{model} = 128$.
- **Learnable Tokens:**

– **CLS Token:** Một vector học được $\mathbf{x}_{cls} \in \mathbb{R}^{1\times d_{model}}$ được chèn vào đầu chuỗi patches. Vector này sẽ tích lũy thông tin toàn cục của ảnh thông qua các tầng Attention:

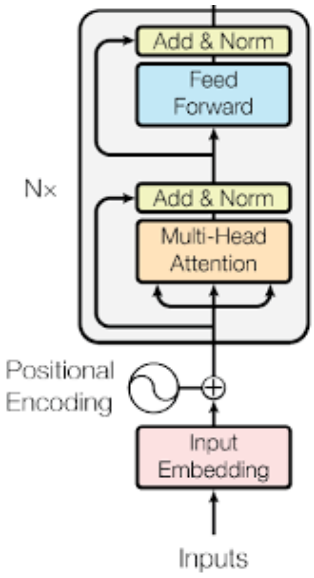
$$\mathbf{x}_0 = [\mathbf{x}_{cls}; \mathbf{x}_{p1}, \mathbf{x}_{p2}, \ldots, \mathbf{x}_{pN}] \in \mathbb{R}^{(N+1)\times d_{model}}$$

– **Position Embedding:** Do cơ chế Self-Attention có tính bất biến với thứ tự (*permutation invariant*), ta cộng thêm một ma trận vị trí học được $E_{pos} \in \mathbb{R}^{(N+1)\times d_{model}}$ để cung cấp thông tin về tọa độ các patch:

$$\mathbf{z}_0 = \mathbf{x}_0 + E_{pos}$$

Dữ liệu đầu ra $\mathbf{z}_0$ *lúc này là một ma trận kích thước* 65×128*, sẵn sàng để đưa vào các khối Transformer Encoder tiếp theo.*

4.2.2 Khối Transformer Encoder tùy chỉnh



Hình 4.1: *Cấu trúc chi tiết một khối Transformer Encoder được hiện thực*

Nhóm đã hiện thực lớp `EncoderLayer` từ các thành phần nguyên tử, tuân thủ cấu trúc **Pre-Norm** để tối ưu hóa dòng chảy gradient (như mô tả trong Hình 4.1):

a. Cơ chế Multi-Head Attention (MHA) với phép toán Einsum

Thay vì sử dụng các hàm thư viện có sẵn, nhóm thực hiện tính toán cơ chế Chú ý tích vô hướng phân tán (*Scaled Dot-Product Attention*) thông qua phép toán `torch.einsum`. Cách tiếp cận này cho phép kiểm soát tường minh các chiều của Tensor và tối ưu hóa hiệu suất tính toán. Các chỉ số trong phép toán được định nghĩa như sau: b (Batch size), h (Số đầu Attention), q/k (Chiều dài chuỗi Token), và d/v (Số chiều đặc trưng của mỗi đầu).

- **Tính toán ma trận tương quan (Attention Score):** Sử dụng `einsum('bhqd, bhkd -> bhqk', Q, K)` để thực hiện nhân ma trận giữa Query (Q) và Key (K).

$$\text{Score}_{bhqk} = \frac{\sum_d (Q_{bhqd} \times K_{bhkd})}{\sqrt{d_k}}$$

Kết quả thu được là một ma trận trọng số thể hiện mức độ liên quan giữa mọi cặp token trong chuỗi ảnh.

- **Tổng hợp thông tin (Context Vector):** Sau khi đi qua hàm *Softmax* để chuẩn hóa các trọng số về dạng xác suất, ta thực hiện phép toán `einsum('bhqk, bhkv -> bhqv', Softmax, V)` để trích xuất đặc trưng từ Value (V).

$$\text{Output}_{bhqv} = \sum_k (\text{Softmax}_{bhqk} \times V_{bhkv})$$

Phép toán này tạo ra vector ngữ cảnh mới, nơi mỗi vị trí token đã được tích hợp thông tin liên quan từ toàn bộ các vị trí khác trong ảnh.

b. Lớp chuẩn hóa LayerNorm tự định nghĩa: Để duy trì tính ổn định của gradient trong các mạng sâu, nhóm đã tự hiện thực lớp `LayerNorm` thủ công thay vì sử dụng `nn.LayerNorm`.

- **Cơ chế:** Lớp này thực hiện chuẩn hóa dữ liệu dựa trên kỳ vọng (μ) và độ lệch chuẩn (σ) tính toán trực tiếp trên chiều cuối cùng (d_{model}) của mỗi token.
- **Tham số học được:** Tích hợp hai tham số γ (gamma) và β (beta) để mô hình có khả năng tái cấu trúc lại phân phối dữ liệu sau chuẩn hóa, giúp tăng tốc độ hội tụ.

c. Feed-Forward Network (FFN): Mỗi khối Encoder kết thúc bằng một mạng FFN nhằm mở rộng khả năng biểu diễn phi tuyến của mô hình. Cấu trúc bao gồm:

- **Tầng mở rộng:** Một lớp tuyến tính đẩy số chiều từ $d_{model} = 128$ lên không gian ẩn $d_{ff} = 256$.
- **Hàm kích hoạt:** Sử dụng `ReLU` để chọn lọc đặc trưng.
- **Tầng thu hẹp:** Một lớp tuyến tính đưa dữ liệu trở lại kích thước $d_{model} = 128$ để chuẩn bị cho khối Encoder tiếp theo.
- **Regularization:** Áp dụng `Dropout` (0.1) sau mỗi tầng để ngăn chặn hiện tượng quá khớp.

4.3 Thiết lập huấn luyện và Benchmark

4.3.1 Thông số cấu hình đồng bộ

Cấu hình thực nghiệm được thiết lập đồng bộ cho cả hai mô hình (*ViT_scratch* và *ViT_builtin*):

- **Cấu hình:** $dim = 128$, $depth = 6$, $num_heads = 8$, $patch_size = 4$ (65 tokens).
- **Tối ưu hóa:** AdamW ($learning_rate = 1e-3$, $weight_decay = 1e-4$) + CosineAnnealingLR.

- **Quy mô huấn luyện:** 20 Epochs; Batch size 64.
- **Tăng cường dữ liệu:** Xoay ngẫu nhiên (15°), lật ngang và cắt ngẫu nhiên (padding=4).
- **Chỉ số đo lường:** Loss, Accuracy, Precision, Recall, F1-score và Thời gian huấn luyện (s).

4.4 Kết quả thực nghiệm và Phân tích

4.4.1 So sánh quy mô tham số

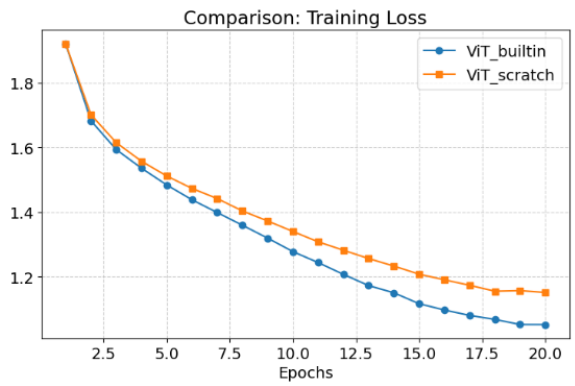
Kiến trúc	Tham số (Triệu)	Trạng thái
ViT_scratch	0.811 M	Manual Implementation
ViT_builtin	0.811 M	PyTorch Optimized

Bảng 4.1: *So sánh số lượng tham số giữa hai phiên bản ViT*

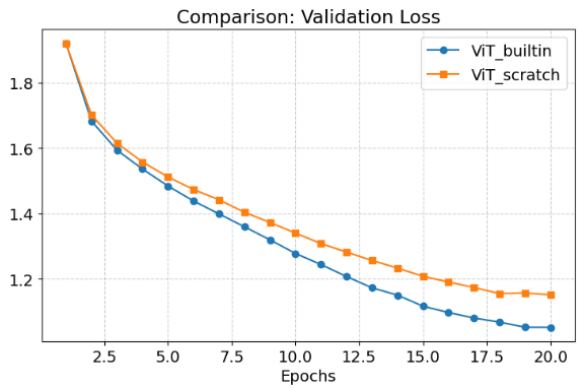
4.4.2 Hiệu năng trên tập kiểm thử (Test Set)

Mô hình	Accuracy	F1-score	Precision	Recall	Time (s)
ViT_builtin	0.6288	0.6254	0.6249	0.6288	951.75
ViT_scratch	0.6069	0.6031	0.6032	0.6069	968.42

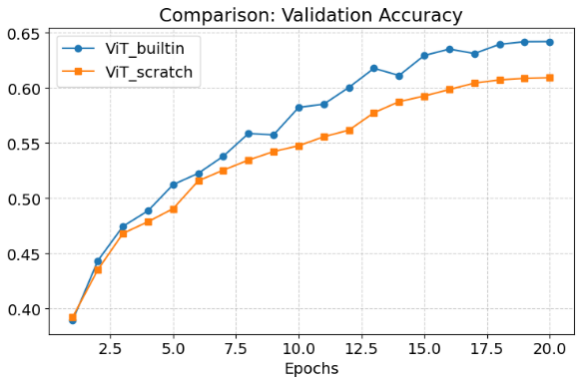
Bảng 4.2: *Kết quả định lượng của hai mô hình sau 20 epoch*



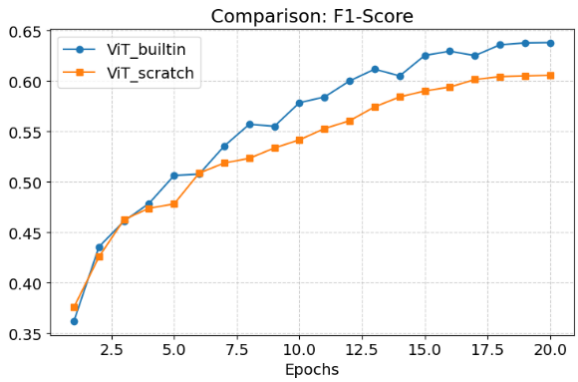
(a) *Training Loss*



(b) *Validation Loss*



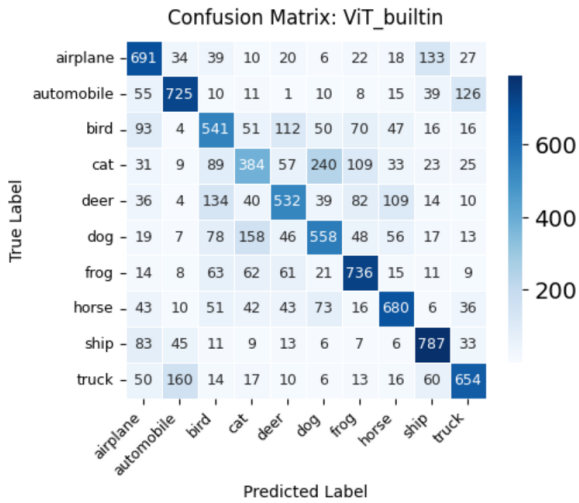
(c) *Validation Accuracy*



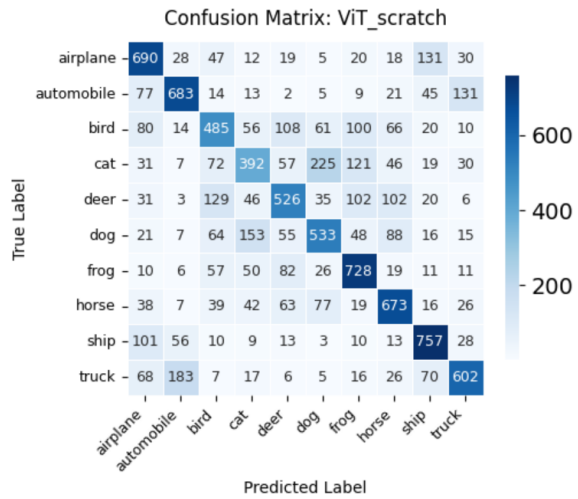
(d) *Validation F1-score*

Hình 4.2: *So sánh lịch sử huấn luyện giữa ViT_scratch và ViT_builtin qua 20 epochs*

Để đánh giá chi tiết khả năng phân loại trên từng lớp đối tượng của CIFAR-10, ma trận nhầm lẫn của hai phiên bản được đối chiếu tại Hình 4.3.



(a) *ViT_builtin confusion matrix*



(b) *ViT_scratch confusion matrix*

Hình 4.3: *Ma trận nhầm lẫn của hai mô hình trên tập Test*

4.4.3 Nhận xét và thảo luận

a. Khả năng hội tụ và độ chính xác:

- **Xu hướng hội tụ:** Quan sát biểu đồ *Training Loss* và *Validation Accuracy*, cả hai mô hình đều cho thấy tốc độ hội tụ ổn định và tương đồng. Điều này chứng minh khối `EncoderLayer` và cơ chế `Pre-Norm` tự hiện thực hoạt động rất tốt, bám sát cấu trúc chuẩn của thư viện.
- **Độ chính xác (Accuracy):** Phiên bản *Built-in* đạt kết quả nhỉnh hơn ($\approx 62.88\%$) so với phiên bản *Scratch* ($\approx 60.69\%$). Chênh lệch khoảng 2% này có thể đến từ các kỹ thuật tối ưu hóa mức thấp trong nhân tính toán của PyTorch và hàm kích hoạt mặc định.

b. Phân tích qua Ma trận nhầm lẫn (Confusion Matrix): Dựa trên hình 4.3, ta thấy hành vi phân loại của hai mô hình có sự tương đồng rõ rệt:

- **Lớp nhận diện tốt:** Cả hai đều đạt độ chính xác cao nhất trên các lớp *Ship*, *Frog* và *Automobile* (đều trên 700 mẫu đúng).
- **Lớp gây nhầm lẫn:** Thách thức lớn nhất nằm ở cặp *Cat* và *Dog*. Cụ thể, *ViT_scratch* nhầm 225 mẫu mèo thành chó, trong khi *ViT_builtin* nhầm 240 mẫu. Sự nhầm lẫn này là đặc trưng của tập dữ liệu CIFAR-10 do độ phân giải thấp (32×32), khiến các đặc trưng thực thể của động vật bốn chân trở nên khó tách biệt đối với kiến trúc Transformer nếu không có pre-training.

c. Hiệu suất tính toán:

- Thời gian huấn luyện của phiên bản tự xây dựng ($968.42s$) chỉ chậm hơn khoảng 1.7% so với thư viện ($951.75s$).
- Điều này khẳng định việc sử dụng phép toán `einsum` giúp tối ưu hóa việc nhân ma trận đa chiều một cách hiệu quả, không gây ra nút thắt cổ chai về mặt hiệu năng tính toán.

4.5 Kết luận

Việc tự hiện thực thành công Transformer Encoder từ các phép toán cơ bản đã giúp hiểu rõ cơ chế lan truyền thông tin và tính toán trọng số Attention. Kết quả thực nghiệm cho thấy mô hình tự xây dựng đạt độ tin cậy cao, là nền tảng vững chắc để thực hiện các cải tiến sâu hơn về kiến trúc.

5

Phần 4 — Các kiến trúc kết hợp và cách embed ảnh khác nhau

5.1 Mục tiêu thực nghiệm

Mục tiêu của phần này là khảo sát các chiến lược biểu diễn token và thiết kế kiến trúc cải tiến trên cùng bài toán CIFAR-10, từ đó đánh giá sự đánh đổi (*trade-off*) giữa khả năng biểu diễn, số lượng tham số và chi phí tính toán. Bài làm triển khai thực nghiệm trên 5 hướng tiếp cận:

- **HybridViT (CNN + Transformer):** Sử dụng các lớp tích chập làm Stem để trích xuất đặc trưng cục bộ trước khi đưa vào Transformer.
- **Overlap Patch Embedding:** Tạo token bằng cửa sổ trượt có sự chồng lấp để tăng cường tính liên kết không gian giữa các patch.
- **Multi-Scale Patch Embedding:** Trích xuất đặc trưng đa tỉ lệ (kết hợp patch 4×4 và 16×16) để học thông tin ở nhiều cấp độ chi tiết.
- **Patch Clustering:** Sử dụng cơ chế gom cụm để tinh lọc và giảm bớt sự dư thừa của các token đầu vào.
- **Grid Patch (Baseline):** Kiến trúc phân mảnh theo lưới tiêu chuẩn để làm cơ sở đối chứng hiệu năng.

5.2 Thiết kế kiến trúc và các phương pháp Biểu diễn Token

5.2.1 Kiến trúc lai HybridViT (CNN + Transformer)

Đây là kiến trúc kết hợp giữa khả năng trích xuất đặc trưng cục bộ của mạng tích chập và khả năng học ngữ cảnh toàn cục của Transformer.

- **CNN Stem:** Thay vì chia ảnh thô ngay lập tức, ảnh đi qua khối `CNNStem` gồm 2 lớp tích chập 3×3 kết hợp với `BatchNorm` và `ReLU`. Lớp đầu tiên giữ nguyên độ phân giải, lớp thứ hai sử dụng `stride=2` để giảm kích thước ảnh xuống 16×16.

- **Cơ chế Tokenization:** Mỗi điểm ảnh trên bản đồ đặc trưng 16×16 được coi là một token (tổng cộng 256 tokens). Chuỗi này sau đó được cộng với mã hóa vị trí (`pos_embedding`) trước khi đưa vào Transformer Encoder.

5.2.2 Các biến thể Patch Embedding (Tokenization)

Nhóm hiện thực ba chiến lược nhúng ảnh khác nhau để đánh giá tầm ảnh hưởng của cách phân mảnh ảnh đến hiệu năng mô hình:

5.2.2.1 Grid Patch Embedding (Ô lưới tiêu chuẩn)

Đây là phương pháp cơ bản của ViT. Ảnh 32×32 được chia thành các ô lưới 4×4 không chồng lấp (`stride = patch_size`).

- **Cấu trúc:** Sử dụng một lớp `Conv2d` với kích thước nhân và bước nhảy đều bằng 4.
- **Kết quả:** Tạo ra chuỗi $8 \times 8 = 64$ tokens. Đây là phương pháp đơn giản nhất với chi phí tính toán thấp.

5.2.2.2 Overlap Patch Embedding (Vùng chồng lấp)

Thay vì chia tách biệt, phương pháp này sử dụng các cửa sổ trượt có kích thước lớn hơn bước nhảy (`stride < patch_size`) để tạo ra các vùng ảnh đè lên nhau.

- **Cấu trúc:** Sử dụng `patch_size=8` và `stride=4`.
- **Đặc điểm:** Việc các patch chồng lấp giúp mô hình nắm bắt được các thông tin liên tục tại ranh giới giữa các vùng ảnh, giúp "mượt hóa"các đặc trưng không gian.

5.2.2.3 Multi-Scale Patch Embedding (Đa tỉ lệ)

Nhóm hiện thực cơ chế trích xuất đặc trưng đa độ phân giải bằng cách sử dụng ba nhánh song song:

- **Scale Fine (4×4):** Lấy chi tiết nhỏ (64 tokens).
- **Scale Medium (8×8):** Lấy đặc trưng tầm trung (16 tokens).
- **Scale Coarse (16×16):** Lấy cấu trúc tổng thể (4 tokens).

Ba luồng thông tin này được nối lại (`torch.cat`) tạo thành một chuỗi 84 tokens, giúp Transformer có cái nhìn toàn diện từ chi tiết đến tổng quát của vật thể.

5.3 Quy trình huấn luyện và Đánh giá

Nhóm thiết lập một quy trình huấn luyện chuẩn hóa cho tất cả các mô hình để đảm bảo tính khách quan trong việc so sánh hiệu năng của các cơ chế Tokenization khác nhau:

5.3.1 Thiết lập thông số huấn luyện (Hyperparameters)

Các mô hình đều được huấn luyện với cùng một cấu hình siêu tham số từ đầu:

- **Số lượng Epoch: 20**
- **Tối ưu hóa (Optimizer):** Sử dụng thuật toán `AdamW` với `learning_rate = 1e-4` và `weight_decay = 1e-4`.
- **Lịch trình học (Scheduler):** Sử dụng `CosineAnnealingLR` giúp mô hình vượt qua các điểm cực tiểu địa phương ở giai đoạn đầu và hội tụ mịn hơn ở giai đoạn cuối.
- **Hàm mất mát:** `CrossEntropyLoss` với `label_smoothing = 0.1`. Việc làm mịn nhãn giúp tăng khả năng tổng quát hóa trên tập Test.

5.3.2 Kiểm soát và Lưu trữ

Nhóm hiện thực lớp `TrainingManager` chuyên biệt để quản lý vòng đời huấn luyện:

- **Cơ chế Early Stopping:** Được thiết lập với `patience = 15`. Nếu độ chính xác trên tập validation không cải thiện trong 15 epochs liên tiếp, tiến trình sẽ tự động dừng để tránh lãng phí tài nguyên tính toán.
- **Lưu trữ trọng số tối ưu:** Hệ thống tự động theo dõi *Validation Accuracy* và chỉ lưu lại file `.pth` của mô hình đạt kết quả cao nhất trong suốt 20 epochs.

5.3.3 Các chỉ số đo lường

Sau khi kết thúc quá trình huấn luyện, mô hình có trọng số tốt nhất sẽ được tải lại để thực hiện đánh giá cuối cùng trên tập Test độc lập. Các chỉ số được ghi nhận bao gồm:

- **Accuracy & F1-score:** Đánh giá khả năng phân loại chính xác và độ tin cậy của mô hình.
- **Training Time:** Tổng thời gian thực thi (giây). Chỉ số này đặc biệt quan trọng để đánh giá tác động của số lượng token (như trong Overlap Patch với 225 tokens) lên tốc độ tính toán của lớp Attention.

5.4 Kết quả thực nghiệm và Phân tích

5.4.1 Bảng thông số cấu trúc và quy mô tham số

Trước khi đánh giá hiệu năng, nhóm thực hiện thống kê quy mô của từng biến thể kiến trúc. Số lượng Token đầu vào đóng vai trò quyết định khối lượng tính toán của tầng Self-Attention ($O(N^2)$).

Mô hình	Số lượng Token	Depth (Layers)	Params	Train Time
HybridViT	**256**	4	640,138	**880.2s**
Grid Patch	64	4	545,930	736.7s
Overlap Patch	49	4	562,442	709.7s
Multi-Scale Patch	84	4	671,626	675.9s

Bảng 5.1: *Thống kê quy mô tài nguyên và chi phí huấn luyện*

5.4.2 Hiệu năng định lượng trên tập dữ liệu

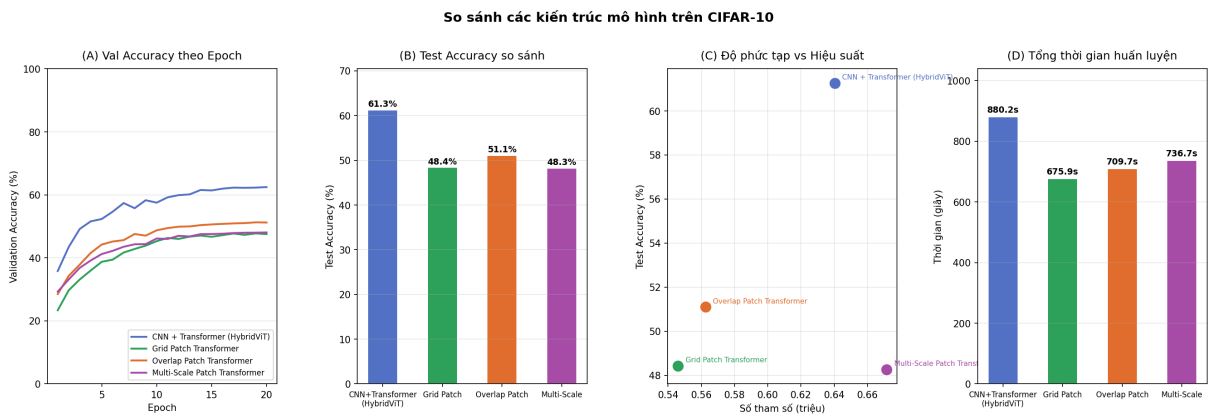
Dưới đây là kết quả thực nghiệm chi tiết thu được sau 20 epochs huấn luyện. Các chỉ số được trích xuất dựa trên bộ trọng số tối ưu nhất (*Best Val Accuracy*) của mỗi mô hình trên tập Test:

Kiến trúc	Val Acc	Test Acc	Precision	Recall	F1-score	Infer
HybridViT	**62.46%**	**61.26%**	**0.6152**	**0.6126**	**0.6094**	1.81s
Overlap Patch	51.27%	51.10%	0.5041	0.5055	0.5019	1.04s
Grid Patch	47.75%	48.42%	0.5120	0.5137	0.5102	1.09s
Multi-Scale Patch	48.06%	48.26%	0.4715	0.4728	0.4682	1.16s

Bảng 5.2: *So sánh đầy đủ các chỉ số hiệu năng giữa các biến thể kiến trúc*

5.4.3 Trực quan hóa kết quả thực nghiệm

Nhóm thực hiện vẽ biểu đồ so sánh dựa trên 4 khía cạnh chủ chốt (Hình 5.1):



Hình 5.1: *Phân tích trực quan từ thực nghiệm: (A) Lịch sử Acc/Loss; (B) So sánh Test Accuracy; (C) Tương quan Params/Acc; (D) Thời gian huấn luyện.*

5.5 Nhận xét và Thảo luận

Dựa trên các bảng số liệu và biểu đồ trực quan, nhóm đưa ra các phân tích chuyên sâu về sự tương quan giữa kiến trúc và hiệu quả học tập:

5.5.1 Sức mạnh của Inductive Bias trong kiến trúc HybridViT

HybridViT đạt độ chính xác **61.26%**, vượt xa nhóm thuần Transformer khoảng 10-13%. Lý do cốt lõi nằm ở đặc tính *Inductive Bias* của CNN Stem: khả năng học đặc trưng cục bộ (cạnh, kết cấu) cực tốt nhờ cơ chế chia sẻ trọng số (*weight sharing*). Điều này thể hiện rõ trên biểu đồ (A): HybridViT tăng trưởng rất dốc trong 5 epoch đầu tiên, trong khi các mô hình thuần túy phải mất nhiều thời gian hơn để tự học cấu trúc không gian từ con số không.

5.5.2 So sánh các chiến lược Patch Embedding

Việc thay đổi cách chia mảnh ảnh mang lại những kết quả khác nhau rõ rệt:

- **Overlap Patch (51.10%):** Đạt kết quả cao nhất trong nhóm thuần Transformer dù số token ít hơn Grid Patch. Việc chồng lấp ranh giới giúp bảo toàn thông tin vật thể khi nó nằm vắt ngang qua nhiều patch, tương tự như cơ chế $stride < kernel_size$ trong CNN.
- **Multi-Scale Patch (48.26%):** Kết quả không vượt qua được Grid Patch, đi ngược lại kỳ vọng lý thuyết. Nhóm nhận định rằng với độ phân giải thấp 32×32, việc sử dụng patch 16×16 (chiếm 1/4 diện tích ảnh) mang lại thông tin ngữ cảnh rất lớn nhưng lại thiếu sự tinh lọc về đặc trưng cục bộ. Thêm vào đó, cơ chế nối (concatenate) đơn giản các token từ các tỉ lệ khác nhau có thể đã gây ra sự nhiễu loạn trong không gian Attention, khiến mô hình khó xác định trọng số ưu tiên giữa các đặc trưng thô và đặc trưng mịn nếu không có một bộ lọc fusion phức tạp hơn

5.5.3 Đánh giá sự đánh đổi (Trade-off) tham số và thời gian

- **Hiệu quả tham số (Efficiency):** Biểu đồ (C) cho thấy Multi-Scale có nhiều tham số nhất nhưng Acc thấp nhất → Efficiency thấp nhất. Ngược lại, HybridViT chiếm vị trí tách biệt, minh chứng rằng một kiến trúc đúng quan trọng hơn việc tăng số lượng tham số đơn thuần.
- **Thời gian thực thi:** HybridViT tốn thời gian huấn luyện nhất (880.2s) do khối lượng tính toán Attention trên 256 tokens. Tuy nhiên, sai lệch về thời gian Inference trên toàn bộ tập test là rất nhỏ (1.04s - 1.81s), cho thấy tất cả các mô hình đều có khả năng triển khai thực tế tốt.

5.6 Kết luận

Thực nghiệm khẳng định **HybridViT** là lựa chọn tối ưu nhất cho bài toán CIFAR-10. Sự kết hợp giữa CNN (trích xuất đặc trưng) và Transformer (học ngữ cảnh) giúp bù đắp khiếm khuyết về lượng dữ liệu nhỏ, tạo ra mô hình có độ chính xác và khả năng hội tụ cao nhất.

6

Phần 5 — Mô hình phân loại dựa trên LSTM/GRU

6.1 Giới thiệu

Trong phần này, nhóm thực hiện một hướng tiếp cận khác bằng cách mô hình hóa dữ liệu hình ảnh 2D thành các cấu trúc dữ liệu tuần tự (Sequential data). Thay vì sử dụng các bộ lọc tích chập cục bộ, nhóm khảo sát khả năng của mạng hồi quy RNN trong việc trích xuất đặc trưng thông qua các phụ thuộc tuần tự không gian (spatial-sequential dependencies). Mục tiêu cốt lõi là kiểm chứng liệu cơ chế ghi nhớ của LSTM và GRU có thể tóm tắt hiệu quả thông tin từ các hàng hoặc cột pixel để phục vụ nhiệm vụ phân loại ảnh hay không. Các mục tiêu thực nghiệm chính bao gồm:

- Triển khai cơ chế chuyển đổi ảnh 2D thành các chuỗi dữ liệu (Row-wise và Column-wise).
- So sánh hiệu năng giữa hai loại đơn vị hồi quy: LSTM và GRU.
- Phân tích ảnh hưởng của hướng quét ảnh (quét theo hàng so với quét theo cột) đến độ chính xác cuối cùng.

6.2 Phương pháp chuyển ảnh thành chuỗi

Để đưa ảnh $3 \times 32 \times 32$ vào mạng RNN, cấu trúc lớp `ImageRNN` đã thực hiện biến đổi dữ liệu theo hai cơ chế:

1. **Row-wise Sequence (Quét theo hàng):** Ảnh được xem như một chuỗi gồm 32 bước thời gian (tương ứng 32 hàng). Tại mỗi bước, mạng nhận vào 32 (pixel) $\times$ 3 (kênh màu) = 96 đặc trưng.
2. **Column-wise Sequence (Quét theo cột):** Ảnh được tráo đổi chiều (*permute*) để quét từ trái sang phải. Chuỗi vẫn gồm 32 bước thời gian, mỗi bước là một cột dọc chứa 96 đặc trưng màu sắc.

Cơ chế này cho phép mạng RNN bức ảnh giống như cách con người quét mắt qua một văn bản, từ đó tóm tắt nội dung vào trạng thái ẩn (*hidden state*) cuối cùng.

6.3 Kiến trúc mô hình

Trong phần này, nhóm xây dựng một kiến trúc mạng hồi quy linh hoạt mang tên `ImageRNN`. Điểm đặc biệt của kiến trúc này là khả năng xử lý dữ liệu ảnh 2D dưới dạng các chuỗi tín hiệu thời gian (time-series), cho phép mô hình học được các phụ thuộc không gian giữa các hàng hoặc các cột ảnh.

6.3.1 Cơ chế biến đổi ảnh thành chuỗi (Image-to-Sequence)

Để đưa ảnh màu CIFAR-10 có kích thước $(B, 3, 32, 32)$ vào mạng RNN, nhóm triển khai hai phương thức quét dữ liệu:

- **Quét theo hàng (Row-wise):** Nhóm xem bức ảnh như một chuỗi gồm 32 bước thời gian. Tại mỗi bước t, mô hình nhận vào một hàng ngang chứa 32 pixel $\times$ 3 kênh màu $= 96$ giá trị đặc trưng.
- **Quét theo cột (Column-wise):** Nhóm sử dụng lệnh `permute` để tráo đổi các chiều của Tensor, biến bức ảnh thành một chuỗi 32 cột dọc. Mỗi bước thời gian vẫn chứa 96 đặc trưng nhưng được trích xuất theo chiều dọc từ trên xuống dưới.

6.3.2 Các biến thể Classifier (LSTM và GRU)

Dựa trên cấu trúc lõi (*RNN Core*), nhóm triển khai hai biến thể mô hình để so sánh:

- **LSTM Classifier (Long Short-Term Memory):** Sử dụng đơn vị nhớ dài hạn với cấu trúc phức tạp gồm 3 loại cổng (Input, Forget, Output) và một trạng thái ô (*cell state*) chạy dọc theo chuỗi thời gian. Cấu trúc này giúp mô hình duy trì thông tin từ những pixel đầu tiên cho đến cuối chuỗi ảnh tốt hơn nhờ khả năng điều tiết luồng thông tin qua các cổng.
- **GRU Classifier (Gated Recurrent Unit):** Sử dụng cấu trúc cổng tối giản hơn bằng cách tích hợp cổng quên và cổng cập nhật thành một cổng duy nhất, đồng thời loại bỏ trạng thái ô riêng biệt. Biến thể này giúp nhóm tối ưu hóa tốc độ tính toán và giảm số lượng tham số cần huấn luyện do có ít ma trận trọng số hơn so với LSTM.

6.3.3 Cấu hình mạng chuyên sâu

- **Cơ chế Bidirectional (RNN hai chiều):** Nhóm thiết lập mạng quét ảnh theo cả hai hướng (ví dụ: từ trên xuống dưới và từ dưới lên trên). Điều này giúp trạng thái ẩn tại mỗi bước thời gian tổng hợp được ngữ cảnh toàn diện hơn.
- **Hệ thống phân loại (Classification Head):** Nhóm thực hiện trích xuất trạng thái ẩn ở mốc thời gian cuối cùng (*Last Hidden State*). Vì RNN có tính chất "nhớ", trạng thái cuối này được xem là bản tóm tắt tinh gọn nhất của toàn bộ bức ảnh. Sau đó, một lớp Tuyến tính (`nn.Linear`) sẽ phóng đại vector 512 chiều (256×2 do dùng hai chiều) thành 10 giá trị xác suất tương ứng với 10 lớp vật thể.

6.4 Thiết lập thực nghiệm

Để đảm bảo tính khách quan và khả năng hội tụ của các mô hình RNN, nhóm đã thiết lập một quy trình thực nghiệm đồng bộ như sau:

6.4.1 Siêu tham số huấn luyện (Hyperparameters)

Nhóm thống nhất bộ tham số cho tất cả các kịch bản thử nghiệm:

Tham số	**Giá trị thiết lập**
Số lượng Epochs	20
Kích thước Batch (Batch Size)	128
Tốc độ học (Learning Rate)	0.001
Hàm tối ưu (Optimizer)	AdamW (Weight Decay = 1e-4)
Lịch trình giảm LR (Scheduler)	CosineAnnealingLR
Số tầng RNN (Num Layers)	2
Kích thước ẩn (Hidden Size)	256 (mỗi chiều)

Bảng 6.1: *Cấu hình siêu tham số trong quá trình thực nghiệm*

6.4.2 Các mô hình so sánh

Để đánh giá toàn diện khả năng của mạng hồi quy trên dữ liệu ảnh, nhóm đã thiết lập 3 thực thể mô hình (scenarios) dựa trên sự thay đổi về kiến trúc lõi và phương thức trích xuất chuỗi:

1. **LSTM_Row (Mô hình chủ đạo):**
   - **Cấu trúc:** Sử dụng lõi LSTM hai chiều (*Bidirectional*).
   - **Phương thức:** Quét ảnh theo từng hàng từ trên xuống dưới.
   - **Mục tiêu:** Đóng vai trò là mô hình cơ sở cao cấp để so sánh với các biến thể khác.
2. **GRU_Row (So sánh kiến trúc lõi):**
   - **Cấu trúc:** Thay thế LSTM bằng lõi GRU hai chiều.
   - **Phương thức:** Giữ nguyên cách quét theo hàng tương tự LSTM_Row.
   - **Mục tiêu:** Đánh giá xem sự tối giản trong cấu trúc cổng của GRU có làm suy giảm khả năng nhận diện vật thể so với LSTM hay không.
3. **LSTM_Col (So sánh hướng quét):**
   - **Cấu trúc:** Sử dụng lõi LSTM hai chiều.
   - **Phương thức:** Thay đổi hướng quét sang quét theo từng cột dọc từ trái sang phải.
   - **Mục tiêu:** Kiểm chứng giả thuyết về tính đối xứng và sự phân bố đặc trưng của vật thể theo chiều ngang/dọc trong bộ dữ liệu CIFAR-10.

6.5 Kết quả thực nghiệm

Trong phần này, nhóm trình bày các số liệu đo lường trực tiếp từ quá trình huấn luyện và kiểm thử các kiến trúc RNN trên bộ dữ liệu CIFAR-10. Các kết quả này phản ánh hiệu năng thực tế của từng mô hình dựa trên các thiết lập đã đề ra.

6.5.1 Hiệu năng định lượng trên tập dữ liệu CIFAR-10

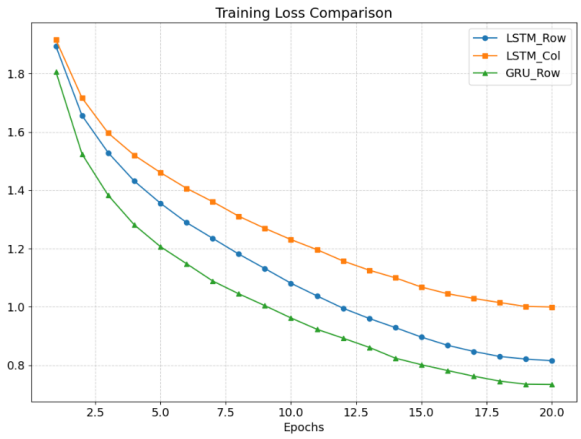
Bảng dưới đây tổng hợp các chỉ số cao nhất thu được sau 20 epochs huấn luyện. Các giá trị được trích xuất tại thời điểm mô hình đạt độ chính xác tốt nhất trên tập Validation:

Mô hình	Best Acc	Best F1	Precision	Recall	Train Time (s)
GRU_Row	**68.22%**	**0.6812**	**0.6811**	**0.6822**	**779.77**
LSTM_Row	63.89%	0.6367	0.6360	0.6389	834.10
LSTM_Col	60.78%	0.6061	0.6055	0.6078	826.45

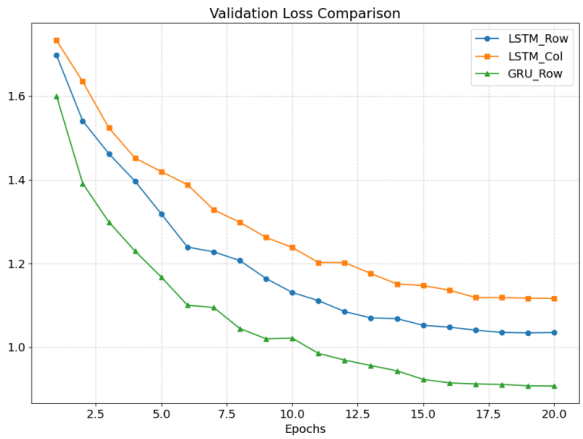
Bảng 6.2: *So sánh hiệu năng chi tiết giữa các kiến trúc RNN*

6.5.2 Trực quan hóa quá trình hội tụ

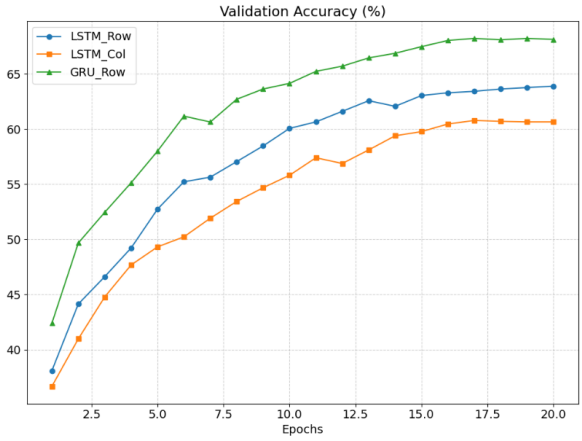
Để đánh giá quá trình hội tụ và tính ổn định của các kiến trúc, nhóm đã trích xuất các biểu đồ theo dõi qua 20 epochs:



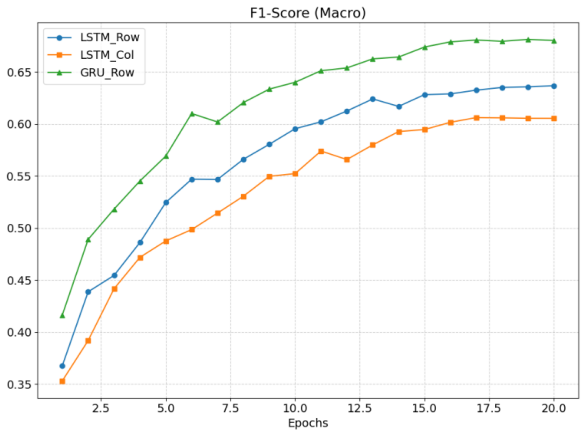
Hình 6.1: *So sánh Training Loss*



Hình 6.2: *So sánh Validation Loss*



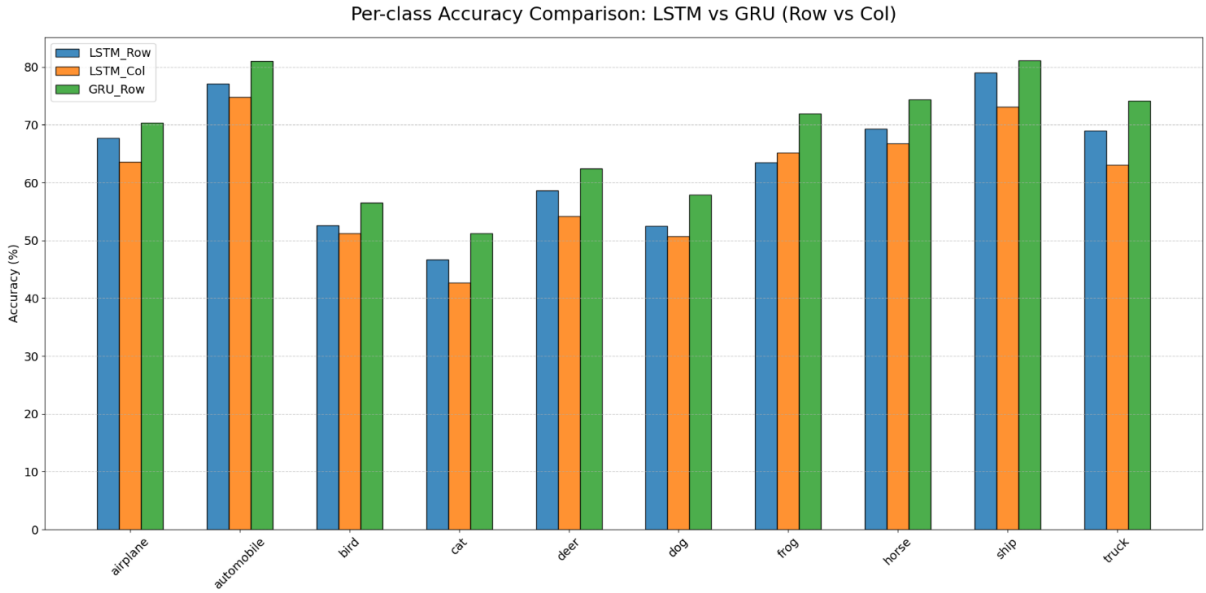
Hình 6.3: *So sánh Validation Accuracy*



Hình 6.4: *Chỉ số F1-Score (Macro)*

6.5.3 So sánh độ chính xác theo từng lớp đối tượng

Để hiểu rõ thế mạnh của từng kiến trúc đối với các loại vật thể khác nhau, nhóm thực hiện trích xuất biểu đồ cột so sánh độ chính xác trên 10 lớp của CIFAR-10:



Hình 6.5: *Biểu đồ so sánh độ chính xác theo từng lớp giữa các biến thể RNN*

6.6 Phân tích và Nhận xét kết quả

6.6.1 Sự vượt trội của kiến trúc GRU

Dựa trên bảng số liệu và biểu đồ độ chính xác, nhóm nhận thấy **GRU_Row** đạt hiệu năng vượt trội nhất với 68.22%.

- **Khả năng hội tụ:** Quan sát biểu đồ *Training Loss*, đường cong của GRU luôn nằm thấp nhất, cho thấy tốc độ học của GRU trên dữ liệu ảnh 32x32 là hiệu quả

nhất.

- **Hiệu năng thời gian:** GRU hoàn thành huấn luyện trong 779.77 giây, nhanh hơn đáng kể so với các biến thể LSTM. Điều này minh chứng cho ưu điểm của cấu trúc cổng tối giản trong GRU giúp giảm khối lượng tính toán nhưng vẫn đạt độ chính xác cao.

6.6.2 Ảnh hưởng của hướng quét (Row-wise vs Column-wise)

Khi so sánh hai biến thể LSTM, nhóm ghi nhận sự chênh lệch rõ rệt:

- Quét theo hàng (**63.89%**) cho kết quả tốt hơn quét theo cột (**60.78%**).
- **Lập luận từ biểu đồ lớp:** Dựa trên Hình 6.5, nhóm nhận thấy các lớp như *"ship"* và *"automobile"* luôn đạt độ chính xác cao nhất (trên 75% với GRU_Row). Do các vật thể này có cấu trúc trải dài theo chiều ngang, việc quét theo hàng giúp mạng nắm bắt sự liên tục của các bộ phận (thân tàu, bánh xe) hiệu quả hơn quét theo cột.

6.6.3 Phân tích sự nhầm lẫn giữa các lớp động vật

Biểu đồ 6.5 cũng chỉ ra rằng các lớp như *"cat"* và *"dog"* có độ chính xác thấp nhất (chỉ quanh mức 45-55%). Nhóm nhận định rằng do RNN học theo chuỗi tuần tự, nó dễ bị đánh lừa bởi các hàng pixel có màu sắc nền tương đồng hoặc vân bề mặt phức tạp của động vật vốn không có cấu trúc hình học ổn định như vật thể nhân tạo.

6.7 Kết luận

Thông qua thực nghiệm, nhóm xác định kiến trúc **GRU kết hợp quét theo hàng (Row-wise)** là giải pháp tối ưu nhất cho bài toán này với độ chính xác đạt 68.22%. Kết quả này chứng minh rằng sự kết hợp đúng đắn giữa hướng quét dữ liệu và một cấu trúc mạng tinh gọn có thể mang lại hiệu suất vượt trội trong xử lý hình ảnh bằng mạng hồi quy.